

2 Sa3yda w 1 Menofy

Competitive Programming Reference

2025

Contents

1 Strategy & Notes	2
1.1 Notes for Hard questions	2
1.2 Checks Before Submission	2
2 Algebra Formulas	2
3 Main Template	3
4 FFT - Templates	3
4.1 FFT (Complex Double)	3
4.2 Mod FFT	4
4.3 NTT (mod = 998244353)	4
4.4 CRT (3-mod combination)	5
5 FFT - Problems	5
5.1 A+B via FFT	5
5.2 A-B Convolution	6
5.3 Poly Shift	6
5.4 Multi Polynomial Multiply	6
5.5 Counting Distinct Array	6
5.6 Product Modulo	6
5.7 String Matching	7
5.8 String Matching WildCard	7
5.9 String Min Mismatch	7
5.10 Fuzzy Search	8
5.11 Max Self Matching	8
6 Data Structures	8
6.1 BIT (Fenwick Tree)	8
6.2 Template BIT	8
6.3 2D BIT	9
6.4 Range BIT	9
6.5 Segment Tree (lazy)	10
6.6 Iterative Segment Tree	10
6.7 Max Subarray Sum Seg Tree	11
6.8 Persistent Segment Tree	12
6.9 Sparse Table	12
6.10 2D Sparse Table	13
6.11 Sqrt Decomposition	13
6.12 DSU	14
6.13 DSU with Rollback	14
6.14 Mo's Algorithm	15

6.15 Mo on Hilbert Order	15
6.16 Monotonic Stack & Queue	15
6.17 2-SAT	16
6.18 Wavelet Tree	17
6.19 Xor Basis	18
7 Graphs	19
7.1 Graph Template	19
7.2 Bellman-Ford	20
7.3 SPFA	21
7.4 MST (Kruskal)	21
7.5 Second Best MST	21
7.6 LCA	21
7.7 LCA with Sparse	22
7.8 Fast LCA	22
7.9 HLD	22
7.10 Centroid Decomposition	23
7.11 DSU on Tree (SACK)	24
7.12 Dsu on Tree	24
7.13 Knapsack on Tree	24
7.14 Block Cut Tree	25
7.15 Tarjan Articulation Points	26
7.16 Tarjan Bridge Tree	26
7.17 Tarjan SCC	26
7.18 Kosaraju	27
7.19 Check for Odd Cycle	27
7.20 Finding Negative Cycle	27
7.21 Max Flow Dinic	28
7.22 MCMF (Dijkstra)	29
7.23 MCMF (SPFA)	30
7.24 Bipartite Matching	31
7.25 Hopcroft-Karp	32
7.26 Hungarian Algorithm	32
8 Strings	33
8.1 KMP Prefix Function	33
8.2 KMP Automaton	33
8.3 KMP Automaton 2	33
8.4 Z-Function	34
8.5 Manacher	34
8.6 Hashing	34
8.7 Shorter Hash	35
8.8 Multiset Hash	35
8.9 Trie	36
8.10 Binary Trie	36
8.11 Aho-Corasick	36

8.12	Aho-Corasick Start/End	39
8.13	Aho Matrix Trick	40
8.14	Hard Hash LCA	43
9	Math & Geometry	46
9.1	Max Manhattan Distance	46
9.2	Linear Sieve	46
9.3	Miller-Rabin Primality	46
9.4	Phi & Mobius Sieve	46
9.5	Phi / Sqrt	46
9.6	Extended Euclidean	47
9.7	Linear Diophantine	47
9.8	Find All Diophantine Solutions	47
9.9	GCD Trick	48
9.10	Count Number of Divisors	49
9.11	Count Subarrays with LCM = q	49
9.12	Counting (Combinations)	49
9.13	Matrix Power	50
9.14	Big Integer	50
9.15	Solve Quadratics	50
9.16	Convex Hull	51
9.17	Geometry	51
10	Dynamic Programming	54
10.1	LIS	54
10.2	LIS with DS	54
10.3	Bitset DP	54
10.4	Digit DP	54
10.5	Digit DP (Kero)	55
10.6	Count Distinct Subsequences	55
10.7	Largest Zero Submatrix	55
10.8	Number of Paths of Length K	56
10.9	Sub-Rects with Unique Values	56
11	Game Theory	57
11.1	Grundy (Sprague-Grundy)	57
11.2	Mex Set	57
12	Miscellaneous	57
12.1	Baby Step Giant Step	57
12.2	Catalan Numbers	57
12.3	Compress	58
12.4	Math Operations on Strings	58
12.5	Ternary Search	59
12.6	2D Partial Sum	59
12.7	Stress Test	59

1 Strategy & Notes

1.1 Notes for Hard questions

- Try to brute force values
- get the naive solution → so you can optimize it
- Stress Test
- Test El PC in Practice
- Think in reverse of the problem
- solve the complement and total - cnt is your answer
- if you need to count the ways and the mean is can be computed easier
- ways = expected * total
- Solve range? try to solve(r) - solve(l - 1)

1.2 Checks Before Submission

- Validate (you ask your teammate)
- Time & Memory
- Corner Cases
- Bounding Cases
- Read Clarifications
- check for mod

2 Algebra Formulas

If a and r are real numbers and $r \neq 0$, then

$$\sum_{j=0}^n ar^j = \begin{cases} \frac{ar^{n+1}-a}{r-1} & \text{if } r \neq 1 \\ (n+1)a & \text{if } r = 1. \end{cases}$$

$$\begin{aligned} \sum_{k=1}^n k &= \frac{n(n+1)}{2} \\ \sum_{k=1}^n k^2 &= \frac{n(n+1)(2n+1)}{6} \\ \sum_{k=1}^n k^3 &= \frac{n^2(n+1)^2}{4} \\ \sum_{k=0}^{\infty} x^k, \|x\| < 1 &= \frac{1}{1-x} \\ \sum_{k=1}^{\infty} kx^{k-1}, \|x\| < 1 &= \frac{1}{(1-x)^2} \end{aligned}$$

- **LCM * GCD** = the multiple of the 2 numbers
- **LCM** = multiply the numbers which has the greatest power from the 2 numbers even if they're not common
- **GCD** = multiply the numbers which has the lowest power from the 2 numbers even if they're not common
- **Arithmetic sequence** : $\frac{n}{2}(a+b)$

- Geometric sequence : $\frac{L*R-a}{r-1} = a * \frac{r^n-1}{r-1}$
- Sum of odd numbers : $\left(\frac{n+n\%2}{2}\right)^2$
- Sum of even numbers: $\frac{(\frac{n}{2}*(2+n-n\%2))}{2}$
- Quadratic Formula : $\frac{-b \pm \sqrt{b^2-4ac}}{2a}$

3 Main Template

```

1 #pragma GCC optimize("O3,unroll-loops")
2 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
3
4 #include <bits/stdc++.h>
5 #include <ext/pb_ds/assoc_container.hpp>
6 #include <ext/pb_ds/tree_policy.hpp>
7
8 #define fast ios_base::sync_with_stdio(0);
9
10 cout.tie(0);
11
12 cin.tie(0)
13
14 using namespace std;
15 using namespace __gnu_pbds;
16 template <typename T>
17 using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
18
19 template <typename T>
20 using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag, tree_order_statistics_node_update>;
21
22 // order_of_key (k) : Number of items strictly smaller than k
23 // find_by_order(k) : K-th element in a set (counting from zero)
24 // const int INF = 1e18;
25
26 // Coding is so easy if you simulate on paper first
27 using lll = __int128_t;
28
29 struct chash
30 {
31     static uint64_t splitmix64(uint64_t x)
32     {
33         x += 0x9e3779b97f4a7c15;
34         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
35         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
36         return x ^ (x >> 31);
37     }
38
39 size_t operator()(uint64_t x) const
40 {
41     static const uint64_t R = chrono::steady_clock::now().time_since_epoch().count();
42     return splitmix64(x + R);
43 }
44
45 size_t operator()(const pair<uint64_t, uint64_t> &p) const
46 {
47     return operator()(p.first) ^ (operator()(p.second) << 1);
48 }

```

```

45     }
46 };
47
48 template <typename K>
49 using hash_set = gp_hash_table<K, null_type, chash>;

```

4 FFT - Templates

4.1 FFT (Complex Double)

```

1 using cd = complex<double>;
2 const double pi = acos(-1);
3
4 void fft(vector<cd> &a, bool invert)
5 {
6     int n = a.size();
7
8     for (int i = 1, j = 0; i < n; i++)
9     {
10         int bit = n >> 1;
11         for (; j & bit; bit >>= 1)
12             j ^= bit;
13         j ^= bit;
14
15         if (i < j)
16             swap(a[i], a[j]);
17     }
18
19     for (int len = 2; len <= n; len <= 1)
20     {
21         double ang = 2 * pi / len * (invert ? -1 : 1);
22         cd wlen(cos(ang), sin(ang));
23         for (int i = 0; i < n; i += len)
24         {
25             cd u = a[i];
26             for (int j = 1; j < len / 2; j++)
27             {
28                 cd v = a[i + j];
29                 cd w = wlen * u;
30                 a[i + j] = u + v;
31                 a[i] = u - v;
32                 w *= wlen;
33             }
34         }
35
36         if (invert)
37         {
38             for (cd &x : a)
39                 x /= n;
40         }
41     }
42 }
43
44 vector<ll> multiply(vector<int> const &a, vector<int> const &b)
45 {
46     vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
47     int n = 1;
48     while (n < (int)a.size() + b.size())
49         n <= 1;

```

```

49 fa.resize(n);
50 fb.resize(n);
51
52 fft(fa, false);
53 fft(fb, false);
54 for (int i = 0; i < n; i++)
55     fa[i] *= fb[i];
56 fft(fa, true);
57
58 vector<ll> result(n);
59 for (int i = 0; i < n; i++)
60     result[i] = round(fa[i].real());
61 return result;
62 }

```

4.2 Mod FFT

```

1 typedef complex<double> C;
2 typedef vector<double> vd;
3
4 void fft(vector<C> &a) {
5     int n = (int)a.size();
6     int L = 31 - __builtin_clz(n);
7
8     static vector<complex<long double>> R(2, 1);
9     static vector<C> rt(2, 1);
10
11     for (static int k = 2; k < n; k *= 2) {
12         R.resize(n);
13         rt.resize(n);
14         auto x = polar(1.0L, acos(-1.0L) / k);
15
16         for (int i = k; i < 2 * k; i++)
17             rt[i] = R[i] = (i & 1) ? R[i / 2] * x : R[i / 2];
18     }
19
20     vector<int> rev(n);
21
22     for (int i = 0; i < n; i++)
23         rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
24
25     for (int i = 0; i < n; i++)
26         if (i < rev[i])
27             swap(a[i], a[rev[i]]);
28
29     for (int k = 1; k < n; k *= 2)
30         for (int i = 0; i < n; i += 2 * k)
31             for (int j = 0; j < k; j++) {
32                 auto x = (double *)&rt[j + k];
33                 auto y = (double *)&a[i + j + k];
34                 C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);
35                 a[i + j + k] = a[i + j] - z;
36                 a[i + j] += z;
37             }
38 }
39
40 template<int M>
41 vector<int> convMod(const vector<int> &a, const vector<int> &b) {
42     if (a.empty() || b.empty()) return {};

```

```

43
44     vector<int> res((int)a.size() + (int)b.size() - 1);
45
46     int B = 32 - __builtin_clz((int)res.size());
47     int n = 1 << B;
48     int cut = (int)sqrt(M);
49
50     vector<C> L(n), R(n), outs(n), outl(n);
51
52     for (int i = 0; i < (int)a.size(); i++)
53         L[i] = C(a[i] / cut, a[i] % cut);
54
55     for (int i = 0; i < (int)b.size(); i++)
56         R[i] = C(b[i] / cut, b[i] % cut);
57
58     fft(L);
59     fft(R);
60
61     for (int i = 0; i < n; i++) {
62         int j = -i & (n - 1);
63         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
64         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
65     }
66
67     fft(outl);
68     fft(outs);
69
70     for (int i = 0; i < (int)res.size(); i++) {
71         long long av = (long long)(real(outl[i]) + .5);
72         long long cv = (long long)(imag(outs[i]) + .5);
73         long long bv = (long long)(imag(outl[i]) + .5) + (long long)(real(outs[i]) + .5);
74
75         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
76     }
77
78     return res;
79 }

```

4.3 NTT (mod = 998244353)

```

1 const int mod = (119 << 23) + 1, root = 3; // = 998244353
2 // For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
3 // and 483 << 21 (same root). The last two are > 10^9.
4
5 int fp(ll b, int e)
6 {
7     ll ans = 1;
8     for (; e; b = b * b % mod, e /= 2)
9         if (e & 1)
10             ans = ans * b % mod;
11     return ans;
12 }
13
14 // Primitive Root of the mod of form 2^a * b + 1
15 int generator()
16 {
17     vector<int> fact;
18     int phi = mod - 1, n = phi;
19     for (int i = 2; i * i <= n; ++i)

```

```

20     if (n % i == 0)
21     {
22         fact.push_back(i);
23         while (n % i == 0)
24             n /= i;
25     }
26     if (n > 1)
27         fact.push_back(n);
28
29     for (int res = 2; res <= mod; ++res)
30     {
31         bool ok = true;
32         for (size_t i = 0; i < fact.size() && ok; ++i)
33             ok &= fp(res, phi / fact[i]) != 1;
34         if (ok)
35             return res;
36     }
37     return -1;
38 }
39
40 void ntt(vector<int> &a)
41 {
42     int n = (int)a.size(), L = 31 - __builtin_clz(n);
43     vector<int> rt(2, 1); // erase the static if you want to use two moduli;
44     for (int k = 2, s = 2; k < n; k *= 2, s++)
45     { // erase the static if you want to use two moduli;
46         rt.resize(n);
47         int z[] = {1, fp(root, mod >> s)};
48         for (int i = k; i < 2 * k; ++i)
49             rt[i] = (ll)rt[i / 2] * z[i & 1] % mod;
50     }
51     vector<int> rev(n);
52     for (int i = 0; i < n; ++i)
53         rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
54     for (int i = 0; i < n; ++i)
55         if (i < rev[i])
56             swap(a[i], a[rev[i]]);
57     for (int k = 1; k < n; k *= 2)
58     {
59         for (int i = 0; i < n; i += 2 * k)
60         {
61             for (int j = 0; j < k; ++j)
62             {
63                 int z = (ll)rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
64                 a[i + j + k] = ai - z + (z > ai ? mod : 0);
65                 ai += (ai + z >= mod ? z - mod : z);
66             }
67         }
68     }
69 }
70 vector<int> conv(const vector<int> &a, const vector<int> &b)
71 {
72     if (a.empty() || b.empty())
73         return {};
74     int s = (int)a.size() + (int)b.size() - 1, B = 32 - __builtin_clz(s), n = 1 << B;
75     int inv = fp(n, mod - 2);
76     vector<int> L(a), R(b), out(n);
77     L.resize(n), R.resize(n);
78     ntt(L), ntt(R);

```

```

79     for (int i = 0; i < n; ++i)
80         out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
81     ntt(out);
82     return {out.begin(), out.begin() + s};
83 }

```

4.4 CRT (3-mod combination)

```

1 int mod, root, desired_mod = 1000000007;
2 const int mod1 = 167772161;
3 const int mod2 = 469762049;
4 const int mod3 = 754974721;
5 const int root1 = 3;
6 const int root2 = 3;
7 const int root3 = 11;
8
9 int CRT(int a, int b, int c, int m1, int m2, int m3) {
10     __int128 M = (__int128)m1*m2*m3;
11     ll M1 = (ll)m2*m3;
12     ll M2 = (ll)m1*m3;
13     ll M3 = (ll)m2*m1;
14
15     int M_1 = modpow(M1%mod1, m1 - 2, m1);
16     int M_2 = modpow(M2%mod2, m2 - 2, m2);
17     int M_3 = modpow(M3%mod3, m3 - 2, m3);
18
19     __int128 ans = (__int128)a*M1*M_1;
20     ans += (__int128)b*M2*M_2;
21     ans += (__int128)c*M3*M_3;
22
23     return (ans % M) % desired_mod;
24 }

```

5 FFT - Problems

5.1 A+B via FFT

```

1 void solve()
2 {
3     int n;
4     cin >> n;
5     vector<int> a(n);
6     int mn = 0;
7     for (int i = 0; i < n; i++)
8         cin >> a[i], mn = min(a[i], mn);
9     const int shift = 5e4;
10    vector<int> p1(4 * shift + 1);
11    int zeros = 0;
12    for (int i = 0; i < n; i++)
13        p1[a[i] + shift]++, zeros += (a[i] == 0);
14    auto pw = multiply(p1, p1);
15    for (int i = 0; i < n; i++)
16        pw[2 * a[i] + 2 * shift]--;
17    int ans = 0;
18    for (int i = 0; i < n; i++)
19    {
20

```

```

21     ans += pw[a[i] + 2 * shift] - 2 * (zeros - (a[i] == 0));
22 }
23 cout << ans << endl;
24 }

```

5.2 A-B Convolution

```

1 vector<int> A_Minus_B(vector<int> a, vector<int> b)
2 {
3     int shift = b.size() - 1;
4     reverse(all(b));
5     // or
6     // for (int i = 0; i < b.size() / 2; i++)
7     //     swap(b[i], b[shift - i]);
8     vector<int> ans = conv(a, b);
9     return {ans.begin() + shift, ans.end()};
10 }

```

5.3 Poly Shift

```

1 vector<int> poly_shift(vector<int> &p, int k)
2 {
3     int deg = p.size() - 1;
4     vector<int> p1(deg + 1), p2(deg + 1);
5     for (int i = 0; i <= deg; i++)
6     {
7         p1[i] = mul(p[i], fact[i]);
8         p2[deg - i] = mul(fp(k, i), ifact[i]);
9     }
10    auto res = conv(p1, p2);
11    vector<int> ans(deg + 1);
12    for(int i = 0; i <= deg; i++)
13        ans[i] = mul(res[i + deg], ifact[i]);
14    return ans;
15 }

```

5.4 Multi Polynomial Multiply

```

1 void solve()
2 {
3     int m;
4     cin >> m;
5     vector<vector<ll>> polys(m + 1);
6     priority_queue<pii, vector<pii>, greater<>> pq; // (size, idx)
7     polys[0] = {1};
8     pq.push({1, 0});
9     for(int i = 1; i <= m; i++)
10    {
11        int n;
12        cin >> n;
13        polys[i].resize(n + 1);
14        for(auto &x : polys[i])
15            cin >> x;
16        pq.push({n + 1, i});
17    }
18    while(pq.size() > 1)
19    {
20        int nIdx = pq.top().second;

```

```

21        auto &p1 = polys[pq.top().second];
22        pq.pop();
23        auto &p2 = polys[pq.top().second];
24        pq.pop();
25        int nSz = p1.size() + p2.size() - 1;
26        p1 = conv(p1, p2);
27        pq.push({nSz, nIdx});
28    }
29    auto &ans = polys[pq.top().second];
30    for(auto &x : ans)
31        cout << x << ' ';
32 }

```

5.5 Counting Distinct Array

```

1 // You have some elements the max ouccurance of each element is m and you want to generate
   // array of size of k of this elements
2 // the answer is fact[k] * sum(ifact of all possible frequencies of each element using ntt or
   // fft with mod)
3
4 void solve()
5 {
6     int n, m;
7     cin >> n >> m;
8     vector<int> p(m + 1);
9     p[m] = fp(fact[m], mod - 2);
10    for (int i = m - 1; i >= 0; i--)
11        p[i] = 1LL * p[i + 1] * (i + 1) % mod;
12    p = poly_pw(p, n);
13    int q;
14    cin >> q;
15    while (q--)
16    {
17        int k;
18        cin >> k;
19        cout << (1LL * fact[k] * p[k] % mod) << ' ';
20    }
21 }

```

5.6 Product Modulo

```

1 // sum (a[i] * b[i] % mod) but sum is not under mod
2
3 const int P = 200003;
4 int to[P];
5 void solve()
6 {
7     ll cur = 1;
8     for (int i = 0; i < P; i++, (cur *= 2) %= P)
9         to[cur] = i;
10    vector<int> p(P);
11    ll sum = 0;
12    int n;
13    cin >> n;
14    for (int i = 0, x; i < n; i++)
15        cin >> x, p[to[x]] += (x > 0), sum -= 1LL * x * x % P;
16    auto ans = multiply(p, p);
17    for (int i = 0; i < ans.size(); i++)
18        if (ans[i])

```

```

19         sum += 1ll * fp(2, i, P) * ans[i];
20     cout << sum / 2 << endl;
21 }

```

5.7 String Matching

```

1 string chars = "abcdefghijklmnopqrstuvwxyz";
2
3 int match_all_chars(const string &T, const string &P)
4 {
5     int n = T.size(), m = P.size();
6     if (m > n)
7         return {};
8
9     int required_matches = 0;
10    for (char c : P)
11    {
12        if (c != '?')
13            required_matches++;
14    }
15
16    vector<int> total_matches(n + m, 0);
17
18    // Loop over all possible alphabet characters
19    for (char c = 'a'; c <= 'z'; c++)
20    {
21        // Skip character if not present in P to save FFT operations
22        bool in_p = false;
23        for (char pc : P)
24        {
25            if (pc == c)
26            {
27                in_p = true;
28                break;
29            }
30        }
31        if (!in_p)
32            continue;
33
34        vector<int> A(n, 0), B(m, 0);
35        for (int i = 0; i < n; i++)
36            if (T[i] == c)
37                A[i] = 1;
38        for (int i = 0; i < m; i++)
39            if (P[m - 1 - i] == c)
40                B[i] = 1;
41
42        vector<ll> C = multiply(A, B);
43        for (int i = 0; i < n + m; i++)
44        {
45            total_matches[i] += C[i];
46        }
47    }
48
49    int cnt = 0;
50    for (int i = 0; i <= n - m; i++)
51        cnt += (total_matches[i + m - 1] == required_matches);
52
53    return cnt;

```

```

54 }

```

5.8 String Matching WildCard

```

1 void solve(int tc) {
2
3     string s, patt; cin >> s >> patt;
4     int n = (int)s.length(), m = (int)patt.length();
5
6     vector<cd> poly1(n), poly2(m);
7
8     for (int i = 0; i < n; ++i) {
9         double angle = 2*PI*(s[i]-'a')/26;
10        poly1[i] = cd(cos(angle), sin(angle));
11    }
12    for (int i = 0; i < m; ++i) {
13        if (patt[m-i-1] == '*') poly2[i] = cd(0,0); // Wild Card
14        else {
15            double angle = 2*PI*(patt[m-i-1]-'a')/26;
16            poly2[i] = cd(cos(angle), -sin(angle));
17        }
18    }
19
20    vector<cd> ans = multiply(poly1, poly2);
21    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');
22
23    int tot = 0;
24    vector<int> pos;
25    for (int i = 0; i < n; ++i) {
26        if (fabs(ans[m-1+i].real() - (m - wild_cnt)) < eps && fabs(ans[m-1+i].imag()) < eps) {
27            ++tot;
28            pos.push_back(i);
29        }
30    }
31
32    cout << tot << "\n";
33    for (auto & p : pos) cout << p << " ";
34    cout << "\n";
35
36 }

```

5.9 String Min Mismatch

```

1 const char chrs[] = {'A', 'C', 'T', 'G'};
2 int stringMinMismatch(string str, string pattern)
3 {
4     int n = str.size();
5     int m = pattern.size();
6     int shift = m - 1;
7     vector<int> ans(n + m + 5);
8     for (auto &ch : chrs)
9     {
10        vector<int> p1(n);
11        vector<int> p2(m);
12        for (int i = 0; i < n; i++)
13            p1[i] = str[i] == ch;
14        for (int i = 0; i < m; i++)
15            p2[shift - i] = pattern[i] == ch;
16        auto mult = multiply(p1, p2);

```

```

17     for (int i = 0; i < n; i++)
18         ans[i + shift] += mult[i + shift];
19 }
20 int mn = m;
21 for (int i = 0; i <= n - m; i++)
22     mn = min(mn, m - ans[i + shift]);
23 return mn;
24 }
25 void solve()
26 {
27     string s, p;
28     cin >> s >> p;
29     cout << stringMinMismatch(s, p) << endl;
30 }

```

5.10 Fuzzy Search

```

1  const string chars = "ATGC";
2
3  void solve()
4  {
5      int n, m, k;
6      cin >> n >> m >> k;
7      string s, t;
8      cin >> s >> t;
9      int shift = m - 1;
10     vector<int> ans(n);
11     for (auto &ch : chars)
12     {
13         vector<int> p1(n + 1), p2(shift + 1);
14         int cnt = 0;
15
16         for (int i = 0; i < n; i++)
17             if (s[i] == ch)
18                 p1[max(0, i - k)]++, p1[min(n, i + k + 1)]--, cnt++;
19         for (int i = 0; i < n; i++)
20             p1[i + 1] += p1[i];
21         p1.pop_back();
22         for (int i = 0; i < n; i++)
23             p1[i] = bool(p1[i]);
24         for (int i = 0; i < m; i++)
25             p2[shift - i] = t[i] == ch;
26         auto mult = multiply(p1, p2);
27         for (int i = 0; i < n; i++)
28             ans[i] += mult[i + shift];
29     }
30     cout << count(all(ans), m) << endl;
31 }

```

5.11 Max Self Matching

```

1  const char chrs[] = {'a', 'b', 'c'};
2  vector<int> stringMaxSelfMatching(string s)
3  {
4      string p = s;
5      int n = s.size();
6      s = s + string(n, '#');
7      int shift = n - 1;
8      vector<int> ans(2 * n + 5);

```

```

9      for(auto &ch : chrs)
10     {
11         vector<int> p1(2 * n);
12         vector<int> p2(n);
13         for(int i = 0; i < 2 * n; i++)
14             p1[i] = s[i] == ch;
15         for(int i = 0; i < n; i++)
16             p2[shift - i] = s[i] == ch;
17         auto mult = multiply(p1, p2);
18         for(int i = 1; i < n; i++)
19             ans[i + shift] += mult[i + shift];
20     }
21     return vector<int>(ans.begin() + shift, ans.begin() + shift + n);
22 }
23 void solve()
24 {
25     string s;
26     cin >> s;
27     auto ans = stringMaxSelfMatching(s);
28     int mx = *max_element(all(ans));
29     cout << mx << endl;
30     for(int i = 1; i < s.size(); i++)
31         if(ans[i] == mx)
32             cout << i << endl;
33 }

```

6 Data Structures

6.1 BIT (Fenwick Tree)

```

1  struct BIT {
2      int n;
3      vector<int> tree;
4
5      BIT(int n) : n(n) {
6          tree.resize(n + 1);
7      }
8
9      void update(int idx, int delta) {
10         for (; idx <= n; idx += (idx & -idx))
11             tree[idx] += delta;
12     }
13
14     int pref(int idx) {
15         int ret = 0;
16         for (; idx > 0; idx -= (idx & -idx))
17             ret += tree[idx];
18         return ret;
19     }
20
21     int get(int l, int r) {
22         return pref(r) - pref(l - 1);
23     }
24 };

```

6.2 Template BIT


```

1  template <typename T>
2  class FenwickTree
3  {
4  public:
5      vector<T> tree;
6      int n;
7      void init(int n)
8      {
9          tree.assign(n + 2, 0);
10         this->n = n;
11     }
12     T merge(T &x, T &y) { return x + y; }
13     void update(int x, T val)
14     {
15         for (; x <= n; x += x & -x)
16         {
17             tree[x] = merge(tree[x], val);
18         }
19     }
20     T getPrefix(int x)
21     {
22         if (x <= 0)
23             return 0;
24         T ret = 0;
25         for (; x; x -= x & -x)
26         {
27             ret = merge(ret, tree[x]);
28         }
29         return ret;
30     }
31 }
32 T getRange(int l, int r)
33 {
34     return getPrefix(r) - getPrefix(l - 1);
35 }
36 int lowerBound(ll x)
37 {
38     int pos = 0;
39     for (int sz = (1 << __lg(n)); sz > 0 && x; sz >>= 1)
40     {
41         if (pos + sz <= n && tree[pos + sz] < x)
42         {
43             x -= tree[pos + sz];
44             pos += sz;
45         }
46     }
47     return pos + 1;
48 }
49 };

```

6.3 2D BIT

```

1  template <typename T>
2  class FenwickTree2D
3  {
4  public:
5      vector<vector<T>> tree;
6      int n, m;
7      void init(int n, int m)

```

```

8      {
9          tree.assign(n + 2, vector<T>(m + 2, 0));
10         this->n = n;
11         this->m = m;
12     }
13     T merge(T &x, T &y) { return x + y; }
14     void update(int x, int y, T val)
15     {
16         for (; x <= n; x += x & -x)
17         {
18             for (int z = y; z <= m; z += z & -z)
19             {
20                 tree[x][z] = merge(tree[x][z], val);
21             }
22         }
23     }
24     T getPrefix(int x, int y)
25     {
26         if (x <= 0)
27             return 0;
28         T ret = 0;
29         for (; x; x -= x & -x)
30         {
31             for (int z = y; z; z -= z & -z)
32             {
33                 ret = merge(ret, tree[x][z]);
34             }
35         }
36         return ret;
37     }
38     T getSquare(int xl, int yl, int xr, int yr)
39     {
40         return getPrefix(xr, yr) + getPrefix(xl - 1, yl - 1) -
41             getPrefix(xr, yl - 1) - getPrefix(xl - 1, yr);
42     }
43 };

```

6.4 Range BIT

```

1  struct BIT
2  {
3      vector<int> tree;
4      vector<int> tree2;
5      int treeSize;
6      BIT(int n) : treeSize(n)
7      {
8          tree.assign(n + 1, 0);
9          tree2.assign(n + 1, 0);
10     }
11     void add(int idx, int delta, vector<int> &tree)
12     {
13         while (idx <= treeSize)
14         {
15             tree[idx] += delta;
16             idx += (idx & -idx);
17         }
18     }
19     int query(int idx, vector<int> &tree)
20     {

```

```

21     int ret = 0;
22     while (idx > 0)
23     {
24         ret += tree[idx];
25         idx -= (idx & -idx);
26     }
27     return ret;
28 }
29 void update(int l, int r, int delta)
30 {
31     add(l, delta, tree);
32     add(r + 1, -delta, tree);
33     add(l, delta * (l - 1), tree2);
34     add(r + 1, -delta * r, tree2);
35 }
36 int pref(int idx)
37 {
38     return query(idx, tree) * idx - query(idx, tree2);
39 }
40 int get(int l, int r)
41 {
42     return pref(r) - pref(l - 1);
43 }
44 };

```

6.5 Segment Tree (lazy)

```

1  struct Node
2  {
3      ll val = 0;
4  };
5  struct SegmentTree
6  {
7      Node neutral;
8      vector<Node> tree;
9      vector<ll> lazy;
10     SegmentTree(int n)
11     {
12         tree.resize(4 * n);
13         lazy.resize(4 * n, 1e6);
14     }
15     Node single(int idx, vector<int> &a)
16     {
17         return Node({a[idx]});
18     }
19     Node merge(Node x, Node y)
20     {
21         return Node({x.val + y.val});
22     }
23     void build(int node, int s, int e, vector<int> &a)
24     {
25         if(s == e)
26         {
27             tree[node] = single(s, a);
28             return;
29         }
30
31         int m = (s + e) >> 1;
32

```

```

33         build(2 * node, s, m, a);
34         build(2 * node + 1, m + 1, e, a);
35         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
36     }
37
38     void propagate(int node, int s, int e)
39     {
40         if(lazy[node] == (ll)1e6)
41             return;
42         if(s == e)
43         {
44             tree[node].val = aa[s + lazy[node]];
45         }
46         if(s != e)
47         {
48             lazy[2 * node] = lazy[node];
49             lazy[2 * node + 1] = lazy[node];
50         }
51         lazy[node] = 1e6;
52     }
53     void update(int node, int s, int e, int l, int r, int val)
54     {
55         propagate(node, s, e);
56         if(s > r || e < l)
57             return;
58         if(s >= l && e <= r)
59         {
60             lazy[node] = val;
61             propagate(node, s, e);
62             return;
63         }
64         int m = (s + e) >> 1;
65
66         update(2 * node, s, m, l, r, val);
67         update(2 * node + 1, m + 1, e, l, r, val);
68         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
69     }
70
71     Node query(int node, int s, int e, int l, int r)
72     {
73         propagate(node, s, e);
74         if(s > r || e < l)
75             return neutral;
76         if(s >= l && e <= r)
77             return tree[node];
78         int m = (s + e) >> 1;
79         return merge(query(2 * node, s, m, l, r), query(2 * node + 1, m + 1, e, l, r));
80     }
81 };

```

6.6 Iterative Segment Tree

```

1  template <typename T>
2  struct SegmentTree
3  {
4      int treeSize;
5      vector<T> tree;
6      T neutral = T();
7      SegmentTree(int n, vector<int> arr = {})

```

```

8   {
9       treeSize = 1;
10      while (treeSize < n)
11          treeSize <= 1;
12      tree.assign((treeSize < 1), neutral);
13      for (int i = 0; i < arr.size(); i++)
14          tree[i + treeSize] = T(arr[i]);
15      for (int i = treeSize - 1; i > 0; i--)
16          tree[i] = tree[i < 1] + tree[i < 1 | 1];
17  }
18  inline void update(int idx, int v) // idx : (0-based)
19  {
20      idx += treeSize;
21      tree[idx].change(v);
22      for (idx >= 1; idx > 0; idx >= 1)
23          tree[idx] = tree[idx < 1] + tree[idx < 1 | 1];
24  }
25  inline int get(int l, int r) // L : (inclusive), R : (exclusive)
26  {
27      T lf, rf;
28      for (l += treeSize, r += treeSize; l < r; l >= 1, r >= 1)
29      {
30          if (l & 1)
31              lf = lf + tree[l++];
32          if (r & 1)
33              rf = tree[--r] + rf;
34      }
35      lf = lf + rf;
36      return lf;
37  }
38  };
39  struct Node
40  {
41      int sum;
42      Node(int sum = INF) : sum(sum)
43      {
44      }
45      inline void change(int sum)
46      {
47          this->sum = sum;
48      }
49      operator int() const
50      {
51          return this->sum;
52      }
53      const Node operator+(const Node &other)
54      {
55          return Node(min(this->sum, other.sum));
56      }
57  };

```

6.7 Max Subarray Sum Seg Tree

```

1  template <typename T>
2  struct SegmentTree
3  {
4      int treeSize;
5      vector<T> tree;
6      T neutral = T();

```

```

7  SegmentTree(int n, vector<int> arr = {})
8  {
9      treeSize = 1;
10     while (treeSize < n)
11         treeSize <= 1;
12     tree.assign((treeSize < 1), neutral);
13     for (int i = 0; i < arr.size(); i++)
14         tree[i + treeSize] = T(arr[i]);
15     for (int i = treeSize - 1; i > 0; i--)
16         tree[i] = tree[i < 1] + tree[i < 1 | 1];
17 }
18 inline void update(int idx, int v) // idx : (0-based)
19 {
20     idx += treeSize;
21     tree[idx].change(v);
22     for (idx >= 1; idx > 0; idx >= 1)
23         tree[idx] = tree[idx < 1] + tree[idx < 1 | 1];
24 }
25 inline int get(int l, int r) // L : (inclusive), R : (exclusive)
26 {
27     T lf, rf;
28     for (l += treeSize, r += treeSize; l < r; l >= 1, r >= 1)
29     {
30         if (l & 1)
31             lf = lf + tree[l++];
32         if (r & 1)
33             rf = tree[--r] + rf;
34     }
35     lf = lf + rf;
36     return lf;
37 }
38 };
39 struct Node
40 {
41     int mx, sum, suff, pref;
42     Node(int sum = 0)
43     {
44         this->sum = sum;
45         sum = max(0ll, sum);
46         this->mx = this->suff = this->pref = sum;
47     }
48     inline void change(int sum)
49     {
50         this->sum = sum;
51         sum = max(0ll, sum);
52         this->mx = this->suff = this->pref = sum;
53     }
54     operator int() const
55     {
56         return this->mx;
57     }
58     const Node operator+(const Node &other)
59     {
60         Node ret;
61         ret.sum = sum + other.sum;
62         ret.pref = max(pref, sum + other.pref);
63         ret.suff = max(other.suff, suff + other.sum);
64         ret.mx = max({
65             mx,

```

```

66         other.mx,
67         suff + other.pref
68     });
69     return ret;
70 }
71 };

```

6.8 Persistent Segment Tree

```

1  struct Node
2  {
3      int sum;
4      // int lx, rx; // lx (inclusive), rx(exclusive)
5      Node *l, *r;
6      Node() : sum(0), l(NULL), r(NULL)
7      {
8      }
9      Node(int sum) : sum(sum), l(NULL), r(NULL)
10     {
11     }
12 };
13 const int R = 1e9 + 1;
14 int mul(int a, int b)
15 {
16     return (a * b) % mod;
17 }
18 void merge(Node *ret)
19 {
20     ret->sum = 1;
21     ret->sum = mul(ret->sum, ret->l ? ret->l->sum : 0);
22     ret->sum = mul(ret->sum, ret->r ? ret->r->sum : 0);
23 }
24 Node *merge(Node *lf, Node *rf)
25 {
26     Node *ret = NULL;
27     if (!lf and !rf)
28         return NULL;
29     ret = new Node();
30     ret->sum = 1;
31     if (lf)
32     {
33         ret->sum = mul(ret->sum, lf->sum);
34         delete lf;
35         lf = NULL;
36     }
37     if (rf)
38     {
39         ret->sum = mul(ret->sum, rf->sum);
40         delete rf;
41         rf = NULL;
42     }
43     return ret;
44 }
45 Node *root;
46 void upd(int idx, int delta, Node *cur, int lx, int rx)
47 {
48
49     if (rx - lx == 1)
50     {

```

```

51         cur->sum += delta;
52         return;
53     }
54     int mid = (lx + rx) / 2;
55     if (idx < mid)
56     {
57         if (!cur->l)
58             cur->l = new Node();
59         upd(idx, delta, cur->l, lx, mid);
60     }
61     else
62     {
63         if (!cur->r)
64             cur->r = new Node();
65         upd(idx, delta, cur->r, mid, rx);
66     }
67     merge(cur);
68 }
69 void upd(int idx, int delta)
70 {
71     upd(idx, delta, root, 0, R);
72 }
73 Node *get(int l, int r, Node *cur, int lx, int rx)
74 {
75     if (lx >= r or rx <= l)
76         return NULL;
77     if (!cur)
78         return new Node(0);
79     if (l <= lx and rx <= r)
80         return new Node(cur->sum);
81     int mid = (lx + rx) / 2;
82     auto lf = get(l, r, cur->l, lx, mid);
83     auto rf = get(l, r, cur->r, mid, rx);
84     auto ret = merge(lf, rf);
85     return ret;
86 }
87 int get(int l, int r)
88 {
89     Node *ret = get(l, r, root, 0, R);
90     if (!ret)
91         return 0;
92     int ret_sum = ret->sum;
93     delete ret;
94     ret = NULL;
95     return ret_sum;
96 }
97 void clear(Node *cur)
98 {
99     if (cur->l)
100         clear(cur->l);
101     if (cur->r)
102         clear(cur->r);
103     delete cur;
104     cur = NULL;
105 }

```

6.9 Sparse Table

```

1  ll myFunction(ll a, ll b){

```

```

2     return a;
3 }
4 struct SparseTable {
5     vector<vector<ll>> table;
6     vector<int> lgs;
7
8     SparseTable(vector<ll>& arr) {
9         int n = arr.size();
10        table = vector<vector<ll>>(n + 1, vector<ll>(20));
11        lgs = vector<int>(n + 1);
12
13        for (int i = 0; i < n; i++)
14            table[i][0] = arr[i];
15        for (int i = 2; i <= n; i++)
16            lgs[i] = lgs[i / 2] + 1;
17        for (int lg = 1; lg <= lgs[n]; lg++)
18            for (int i = 0; i + (1ll << lg) - 1 < n; i++)
19                table[i][lg] =
20                    myFunction(table[i][lg - 1], table[i + (1ll << (lg - 1))][lg - 1]);
21    }
22
23    ll get(int l, int r) {
24        return myFunction(table[l][lgs[r - l + 1]],
25                           table[r - (1 << lgs[r - l + 1]) + 1][lgs[r - l + 1]]);
26    }
27 };

```

6.10 2D Sparse Table

```

1 int g[N][N];
2 struct SparseTable2D
3 {
4     int lg2[N];
5     int st[N][N][LG][LG];
6     void pre(int n, int m)
7     {
8         for (int i = 2; i < N; i++)
9             lg2[i] = lg2[i >> 1] + 1;
10        for (int i = 0; i < n; i++)
11        {
12            for (int j = 0; j < m; j++)
13            {
14                st[i][j][0][0] = g[i][j];
15            }
16        }
17    }
18    void build(int n, int m)
19    {
20        pre(n, m);
21        for (int a = 0; a < LG; a++)
22        {
23            for (int b = 0; b < LG; b++)
24            {
25                if (a + b == 0)
26                    continue;
27                for (int i = 0; i + (1 << a) <= n; i++)
28                {
29                    for (int j = 0; j + (1 << b) <= m; j++)
30                    {

```

```

31                if (!a)
32                    st[i][j][a][b] = max(st[i][j][a][b - 1],
33                                           st[i][j + (1 << (b - 1))][a][b - 1]);
34
35                else
36                    st[i][j][a][b] = max(st[i][j][a - 1][b], st[i
37                                           + (1 << (a - 1))][j][a
38                                           - 1][b]);
39            }
40        }
41    }
42 }
43 }
44 }
45 int query(int x1, int y1, int x2, int y2)
46 {
47     x2++, y2++;
48     int a = lg2[x2 - x1], b = lg2[y2 - y1];
49
50     return max(
51         max(st[x1][y1][a][b], st[x2 - (1 << a)][y1][a][b]),
52         max(st[x1][y2 - (1 << b)][a][b], st[x2 - (1 << a)][y2 -
53                                           (1 << b)][a][b]));
54 }
55 }
56 };

```

6.11 Sqrt Decomposition

```

1 const int sqt = 450;
2 int n{0}, q{0};
3 int lazy[sqt + 1];
4 int lazySet[sqt + 1];
5 int buckets[sqt + 1];
6 int arr[N];
7 void build(int buc_num)
8 {
9     buckets[buc_num] = 0;
10    for (int i = buc_num * sqt; i < min(n, (buc_num + 1) * sqt); i++)
11    {
12        if (lazySet[buc_num])
13            arr[i] = lazySet[buc_num];
14        if (lazy[buc_num])
15            arr[i] += lazy[buc_num];
16        buckets[buc_num] += arr[i];
17    }
18    lazySet[buc_num] = lazy[buc_num] = 0;
19 }
20 void updateSet(int l, int r, int x)
21 {
22     build(l / sqt);
23     build(r / sqt);
24     for (int i = l; i <= r; i++)
25     {
26         if (i % sqt == 0 and i + sqt - 1 <= r)
27         {
28             buckets[i / sqt] = sqt * x;
29             lazy[i / sqt] = 0;

```

```

30     lazySet[i / sqt] = x;
31     i += sqt;
32 }
33 else
34 {
35     buckets[i / sqt] -= arr[i];
36     arr[i] = x;
37     buckets[i / sqt] += arr[i];
38     i++;
39 }
40 }
41 };
42 void update(int l, int r, int x)
43 {
44     build(l / sqt);
45     build(r / sqt);
46
47     for (int i = l; i <= r;)
48     {
49         if (i % sqt == 0 and i + sqt - 1 <= r)
50         {
51             buckets[i / sqt] += sqt * x;
52             lazy[i / sqt] += x;
53             i += sqt;
54         }
55         else
56         {
57             buckets[i / sqt] -= arr[i];
58             arr[i] += x;
59             buckets[i / sqt] += arr[i];
60             i++;
61         }
62     }
63 };
64 int query(int l, int r)
65 {
66     build(l / sqt);
67     build(r / sqt);
68     int ret = 0;
69     for (int i = l; i <= r;)
70     {
71         if (i % sqt == 0 and i + sqt - 1 <= r)
72         {
73             ret += buckets[i / sqt];
74             i += sqt;
75         }
76         else
77         {
78             ret += arr[i];
79             i++;
80         }
81     }
82     return ret;
83 }

```

6.12 DSU

```

1 struct DSU
2 {

```

```

3     vector<int> par, sz;
4     DSU(int n)
5     {
6         sz = par = vector<int>(n + 1, 1);
7         iota(par.begin(), par.end(), 0);
8     }
9     void merge(int u, int v)
10    {
11        u = getPar(u);
12        v = getPar(v);
13        if (u == v)
14            return;
15        if (sz[v] > sz[u])
16            swap(v, u);
17        par[v] = u;
18        sz[u] += sz[v];
19    }
20    int getPar(int u)
21    {
22        if (par[u] == u)
23            return u;
24        return par[u] = getPar(par[u]);
25    }
26    bool get(int u, int v)
27    {
28        return (getPar(u) == getPar(v));
29    }
30 };

```

6.13 DSU with Rollback

```

1 struct DSU
2 {
3     vector<int> par, sz;
4     stack<pair<int &, int>> rollbacks;
5     DSU(int n)
6     {
7         par = sz = vector<int>(n + 1, 1);
8         iota(all(par), 0);
9     }
10    int getPar(int u)
11    {
12        if (par[u] == u)
13            return u;
14        return getPar(par[u]); // No path compression due to rollbacks
15    }
16    void merge(int u, int v)
17    {
18        u = getPar(u);
19        v = getPar(v);
20        rollbacks.push({par[u], par[u]});
21        rollbacks.push({par[v], par[v]});
22        rollbacks.push({sz[u], sz[u]});
23        rollbacks.push({sz[v], sz[v]});
24        if (u == v)
25            return;
26        if (sz[v] > sz[u])
27            swap(v, u);
28        sz[u] += sz[v];

```

```

29     par[v] = u;
30 }
31 void rollback()
32 {
33     assert(rollbacks.size() >= 4);
34     for (int i = 0; i < 4; i++)
35     {
36         auto [variable, value] = rollbacks.top();
37         variable = value;
38         rollbacks.pop();
39     }
40 }
41 int get(int u, int v)
42 {
43     return getPar(u) == getPar(v);
44 }
45 };

```

6.14 Mo's Algorithm

```

1  const int sqt = 320;
2  struct query
3  {
4      int l, r, i;
5      bool operator<(query &other)
6      {
7          int b1 = l / sqt;
8          int b2 = other.l / sqt;
9          if(b1 != b2) return b1 < b2;
10         return r < other.r;
11     }
12 };
13
14 int freq[N]{};
15 int n, q;
16 int answer = 0;
17 int arr[N]{};
18 void add(int idx)
19 {
20     int val = arr[idx];
21     if(val > N)
22         return;
23     answer -= (freq[val] == val);
24     freq[val]++;
25     answer += (freq[val] == val);
26 }
27 void rem(int idx)
28 {
29     int val = arr[idx];
30     if(val > N)
31         return;
32
33     answer -= (freq[val] == val);
34     freq[val]--;
35     answer += (freq[val] == val);
36 }
37
38 void MosAlgo(vector<query> &queries, vector<int> &ans)
39 {

```

```

40     sort(all(queries));
41     int l = 0, r = -1;
42     for(auto &[L, R, idx] : queries)
43     {
44         while (r < R) add(++r);
45         while(l > L) add(--l);
46         while(r > R) rem(r--);
47         while(l < L) rem(l--);
48         ans[idx] = answer;
49     }
50 }

```

6.15 Mo on Hilbert Order

```

1  inline int64_t hilbertOrder(int x, int y, int pow, int rotate)
2  {
3      if (pow == 0)
4      {
5          return 0;
6      }
7      int hpow = 1 << (pow - 1);
8      int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
9      seg = (seg + rotate) & 3;
10     const int rotateDelta[4] = {3, 0, 0, 1};
11     int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
12     int nrot = (rotate + rotateDelta[seg]) & 3;
13     int64_t subSquareSize = int64_t(1) << (2 * pow - 2);
14     int64_t ans = seg * subSquareSize;
15     int64_t add = hilbertOrder(nx, ny, pow - 1, nrot);
16     ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
17     return ans;
18 }

```

6.16 Monotonic Stack & Queue

```

1  template <class T>
2  struct Mono_stack
3  {
4      stack<pair<T, T>> st;
5      void push(const T &val)
6      {
7          if (st.empty())
8              st.emplace(val, val);
9          else
10             st.emplace(val, std::max(val, st.top().second));
11     }
12     void pop()
13     {
14         st.pop();
15     }
16     bool empty()
17     {
18         return st.empty();
19     }
20     int size()
21     {
22         return st.size();
23     }
24     T top()

```

```

25     {
26         return st.top().first;
27     }
28     T max()
29     {
30         return st.top().second;
31     }
32 };
33 template <class T>
34 struct Mono_queue
35 {
36     Mono_stack<T> pop_st, push_st;
37     void push(const T &val)
38     {
39         push_st.push(val);
40     }
41     void move()
42     {
43         if (pop_st.size())
44             return;
45         while (!push_st.empty())
46             pop_st.push(push_st.top(), push_st.pop());
47     }
48     void pop()
49     {
50         move();
51         pop_st.pop();
52     }
53     bool empty()
54     {
55         return pop_st.empty() && push_st.empty();
56     }
57     int size()
58     {
59         return pop_st.size() + push_st.size();
60     }
61     T top()
62     {
63         move();
64         return pop_st.top();
65     }
66     T max()
67     {
68         if (pop_st.empty())
69             return push_st.max();
70         if (push_st.empty())
71             return pop_st.max();
72         return std::max(push_st.max(), pop_st.max());
73     }
74 };

```

6.17 2-SAT

```

1  int n, m;
2  vector<int> a[N];
3  int dfsn[N], lowLink[N], inStack[N], comp[N], repr[N], dtime, scc;
4  stack<int> st;
5  void tarjan(int node)
6  {

```

```

7      dfsn[node] = lowLink[node] = dtime++, inStack[node] = 1;
8      st.push(node);
9      for (auto &i : a[node])
10     {
11         if (dfsn[i] == -1)
12         {
13             tarjan(i);
14             lowLink[node] = min(lowLink[node], lowLink[i]);
15         }
16         else if (inStack[i])
17             lowLink[node] = min(lowLink[node], dfsn[i]);
18     }
19     if (dfsn[node] == lowLink[node])
20     {
21         int x = -1;
22         while (x != node)
23         {
24             x = st.top(), st.pop();
25             inStack[x] = 0;
26             comp[x] = scc;
27         }
28         repr[scc] = node;
29         scc++;
30     }
31 }
32 int NOT(int x)
33 {
34     return 2 * m - x + 1;
35 }
36 void add(int x, int y)
37 {
38     a[NOT(x)].push_back(y);
39     a[NOT(y)].push_back(x);
40 }
41 void doWork()
42 {
43     cin >> n >> m;
44     for (int i = 0; i < n; i++)
45     {
46         char op;
47         int x, y;
48         cin >> op >> x;
49         if (op == '-')
50             x = NOT(x);
51         cin >> op >> y;
52         if (op == '-')
53             y = NOT(y);
54         add(x, y);
55     }
56     memset(dfsn, -1, sizeof dfsn);
57     for (int i = 1; i <= 2 * m; i++)
58     {
59         if (dfsn[i] == -1)
60             tarjan(i);
61     }
62     for (int i = 1; i <= m; i++)
63     {
64         if (comp[i] == comp[NOT(i)])
65         {

```



```

66         cout << "IMPOSSIBLE";
67         return;
68     }
69 }
70 vector<int> comp_values(scc + 1, -1), ans(m + 1);
71 for (int i = 0; i < scc; i++)
72 {
73     if (comp_values[i] == -1)
74     {
75         comp_values[i] = 1;
76         int dual = comp[NOT(repr[i])];
77         comp_values[dual] = 0;
78     }
79 }
80 for (int i = 1; i <= m; i++)
81     ans[i] = comp_values[comp[i]];
82 }

```

6.18 Wavelet Tree

```

1 // Wavelet Tree – O(n log σ) build, O(log σ) query
2 // Positions 1-indexed [L,R]. Build: WT(arr+1, arr+n+1, lo, hi)
3 //
4 // Derivable (not in code):
5 //   lt(L,R,v)           = lte(L,R,v-1)
6 //   range_count(L,R,a,b) = lte(L,R,b) - lte(L,R,a-1)
7 //   range_sum(L,R,a,b)   = sum_lte(L,R,b) - sum_lte(L,R,a-1)
8 //   sum_k(L,R,k)         = sum_lte(L,R, kth(L,R,k))
9 //                         - kth(L,R,k) * (lte(L,R,kth(L,R,k)) - k)
10 //   prev_value(L,R,v)    = kth(L,R, lte(L,R,v)) - 1 if lte==0
11 //   next_value(L,R,v)    = kth(L,R, lte(L,R,v-1)+1) - 1 if lte==R-L+1
12 struct WT {
13     int lo, hi;
14     vector<int> b;
15     vector<ll> t; // b[i]=prefix count left, t[i]=prefix sum left
16     WT *l = NULL, *r = NULL;
17
18     WT(int *from, int *to, int x, int y) {
19         lo = x, hi = y;
20         if (lo == hi || from == to)
21             return;
22         int mid = lo + (hi - lo) / 2;
23         int n = to - from;
24         b.assign(n + 1, 0);
25         t.assign(n + 1, 0);
26         for (int i = 0; i < n; i++) {
27             b[i + 1] = b[i] + (from[i] <= mid);
28             t[i + 1] = t[i] + (from[i] <= mid ? from[i] : 0);
29         }
30         auto f = [mid](int v) { return v <= mid; };
31         auto pivot = stable_partition(from, to, f);
32         if (from != pivot)
33             l = new WT(from, pivot, lo, mid);
34         if (pivot != to)
35             r = new WT(pivot, to, mid + 1, hi);
36     }
37
38     // # values <= k in [L,R]
39     int lte(int L, int R, int k) {

```

```

40         if (L > R || k < lo)
41             return 0;
42         if (hi <= k)
43             return R - L + 1;
44         int lb = b[L - 1], rb = b[R];
45         int mid = lo + (hi - lo) / 2;
46         if (k <= mid) {
47             if (!l)
48                 return l->lte(lb + 1, rb, k);
49             return 0;
50         }
51         int left_count = rb - lb;
52         if (!r)
53             return left_count + r->lte(L - lb, R - rb, k);
54         return left_count;
55     }
56
57     // k-th smallest in [L,R], 1-indexed
58     int kth(int L, int R, int k) {
59         if (lo == hi)
60             return lo;
61         int lb = b[L - 1], rb = b[R], inL = rb - lb;
62         if (k <= inL)
63             return l->kth(lb + 1, rb, k);
64         return r->kth(L - lb, R - rb, k - inL);
65     }
66
67     // sum of values <= v in [L,R]
68     ll sum_lte(int L, int R, int v) {
69         if (L > R || v < lo)
70             return 0;
71
72         // Base case: leaf node
73         if (lo == hi) {
74             return 1LL * (R - L + 1) * lo;
75         }
76
77         int lb = b[L - 1], rb = b[R];
78         int mid = lo + (hi - lo) / 2;
79
80         // Traverse left
81         if (v <= mid) {
82             if (!l)
83                 return l->sum_lte(lb + 1, rb, v);
84             return 0;
85         }
86
87         // Traverse right (this naturally handles the hi <= v case)
88         ll left_sum = t[R] - t[L - 1];
89         if (!r)
90             return left_sum + r->sum_lte(L - lb, R - rb, v);
91         return left_sum;
92     }
93
94     // # distinct values in [L,R]
95     int count_distinct(int L, int R) {
96         if (L > R)
97             return 0;
98         if (lo == hi)

```

```

99     return 1;
100     int lb = b[L - 1], rb = b[R];
101     int res = 0;
102     if (lb < rb)
103         res += l->count_distinct(lb + 1, rb);
104     if (L - lb <= R - rb)
105         res += r->count_distinct(L - lb, R - rb);
106     return res;
107 }
108
109 ~WT() {
110     delete l;
111     delete r;
112 }
113 };

```

6.19 Xor Basis

```

1 struct XB
2 {
3     static constexpr int B = 60;
4     array<ll, B + 1> base{};
5     int rank = 0;
6     XB(int val = 0)
7     {
8         if (val)
9             insert(val);
10    }
11    void insert(ll x)
12    {
13        for (int i = B; i >= 0; --i)
14        {
15            if (((x >> i) & 1LL) == 0)
16                continue;
17            if (base[i] == 0)
18            {
19                base[i] = x;
20                ++rank;
21                return;
22            }
23            x ^= base[i];
24        }
25    }
26
27    // Check if x can be formed as XOR of some inserted numbers
28    bool canRepresent(ll x) const
29    {
30        for (int i = B; i >= 0; --i)
31        {
32            if (((x >> i) & 1LL) && base[i])
33                x ^= base[i];
34        }
35        return x == 0;
36    }
37
38    // Maximum XOR value achievable from any subset (starting from 0 by default)
39    ll getMax(ll start = 0) const
40    {
41        ll res = start;

```

```

42        for (int i = B; i >= 0; --i)
43        {
44            if (!base[i])
45                continue;
46            res = max(res, res ^ base[i]);
47        }
48        return res;
49    }
50    ll getKth(ll k)
51    {
52        if (k > (1LL << rank))
53            return -1;
54        k--;
55        ll res = 0;
56        for (int i = B, j = rank - 1; i >= 0; --i)
57        {
58            if (!base[i])
59                continue;
60            if ((k >> j) & 1)
61                res = max(res, res ^ base[i]);
62            else
63                res = min(res, res ^ base[i]);
64            j--;
65        }
66        return res;
67    }
68
69    int getRank() const
70    {
71        return rank;
72    }
73    const XB operator+(const XB &other) const
74    {
75        XB ret = *this;
76        for (int i = B; i >= 0; --i)
77        {
78            if (other.base[i])
79                ret.insert(other.base[i]);
80        }
81        return ret;
82    }
83 };
84
85 // Xor Basis in Range (Offline precompute)
86
87
88
89 constexpr int B = 60;
90 ll basis[B + 1];
91 int closest[B + 1];
92 void insert(ll x, int idx)
93 {
94     for (int i = B; i >= 0; i--)
95     {
96         if (!(x >> i & 1))
97             continue;
98         if (closest[i] < idx)
99         {

```

100

```

101         swap(x, basis[i]);
102         swap(closest[i], idx);
103     }
104     x ^= basis[i];
105 }
106 }
107 bool canRepresent(int l, ll x)
108 {
109     for (int i = B; i >= 0; i--)
110     {
111         if (!(x >> i & 1))
112             continue;
113         if (closest[i] < l)
114             return false;
115         x ^= basis[i];
116     }
117     return true;
118 }
119
120 // Xor Basis with Bitset
121
122 constexpr int B = 1504;
123
124 using big = bitset<B + 1>;
125 big basis[B + 1];
126 int ids[B + 1];
127 big msk[B + 1];
128 const big one = 1;
129 bool insert(big x, int id)
130 {
131     big mask = 0;
132     for (int i = B; i >= 0; i--)
133     {
134         if (!x[i])
135             continue;
136         if (basis[i] == 0)
137         {
138             basis[i] = x;
139             ids[i] = id;
140             msk[i] = mask ^ (one << i);
141             return true;
142         }
143         mask ^= msk[i];
144         x ^= basis[i];
145     }
146     return false;
147 }
148 int n, k;
149 vector<int> can(big x)
150 {
151     big mask = 0;
152     for (int i = B; i >= 0; i--)
153     {
154         if (x[i] and basis[i] != 0)
155             x ^= basis[i], mask ^= msk[i];
156     }
157     if (x != 0 or mask.count() == n)
158         return {-1};
159     vector<int> ret;

```

```

160     for (int i = B; i >= 0; i--)
161         if (mask[i])
162             ret.push_back(ids[i]);
163     return ret;
164 }

```

7 Graphs

7.1 Graph Template

```

1 bool visited[N]{};
2 void dfs(vector<vector<int>> &graph, int st)
3 {
4     visited[st] = true;
5     for (auto &child : graph[st])
6         if (!visited[child])
7             dfs(graph, child);
8 }
9
10 vector<int> bfs(vector<vector<int>> &graph, int st)
11 {
12     vector<int> dist(graph.size(), INF);
13     queue<int> nextToVisit;
14     dist[st] = 0;
15     nextToVisit.push(st);
16     while (!nextToVisit.empty())
17     {
18         auto cur = nextToVisit.front();
19         nextToVisit.pop();
20         for (auto &ch : graph[cur])
21         {
22             if (dist[ch] == INF)
23             {
24                 dist[ch] = dist[cur] + 1;
25                 nextToVisit.push(ch);
26             }
27         }
28     }
29     return dist;
30 }
31
32 vector<int> dijk(vector<vector<pii>> &graph, int st)
33 {
34     vector<int> dist(graph.size(), INF);
35     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>
36     nextToVisit;
37     dist[st] = 0;
38     nextToVisit.push({0, st});
39     while (nextToVisit.size())
40     {
41         auto [curCost, curDist] = nextToVisit.top();
42         nextToVisit.pop();
43         if (curCost > dist[curDist])
44             continue;
45         for (auto &[cw, cd] : graph[curDist])
46         {
47             if (dist[cd] > cw + curCost)

```

```

48         dist[cd] = cw + curCost;
49         nextToVisit.push({dist[cd], cd});
50     }
51 }
52 }
53 return dist;
54 }
55
56 void kahn(vector<vector<int>> &graph, int &n)
57 {
58     vector<int> degree(n + 1, 0);
59     vector<bool> visited(n + 1, false);
60     for (int i = 1; i <= n; i++)
61     {
62         for (auto &j : graph[i])
63         {
64             degree[j]++;
65         }
66     }
67     queue<int> nextToVisit;
68     for (int i = 1; i <= n; i++)
69     {
70         if (!degree[i])
71         {
72             nextToVisit.push(i);
73             visited[i] = true;
74         }
75     }
76     vector<int> result;
77     while (nextToVisit.size())
78     {
79         int cur = nextToVisit.front();
80         nextToVisit.pop();
81         result.push_back(cur);
82         for (auto &j : graph[cur])
83         {
84             degree[j]--;
85             if (!degree[j])
86                 nextToVisit.push(j);
87         }
88     }
89     if (result.size() != n)
90     {
91         cout << "IMPOSSIBLE" << endl;
92     }
93     else
94     {
95         for (auto &j : result)
96             cout << j << ' ';
97         cout << endl;
98     }
99 }
100
101 vector<int> BellmanFord(vector<vector<pii>> &graph, int &V, int st)
102 {
103     vector<int> dist(V + 1, INF);
104     dist[st] = 0;
105     bool flag = false;
106     for (int edgesUsed = 1; edgesUsed <= V; edgesUsed++)

```

```

107 {
108     for (int u = 1; u <= V; u++)
109     {
110         if (dist[u] == INF)
111             continue;
112         for (auto &[cost, v] : graph[u])
113         {
114             if (dist[v] > dist[u] + cost)
115                 dist[v] = dist[u] + cost, flag |= (v == V and edgesUsed == V);
116         }
117     }
118 }
119 if(flag)
120     return {};
121 return dist;
122 }
123 void floyd()
124 {
125     int d[N][N];
126     for(int k = 1; k <= n; k++)
127         for(int u = 1; u <= n; u++)
128             for(int v = 1; v <= n; v++)
129                 d[u][v] = max(d[u][v], d[u][k] + d[k][v]);
130 }

```

7.2 Bellman-Ford

```

1 struct Edge {
2     int u, v;
3     long long w;
4 };
5
6 void solve() {
7     int n, m;
8     cin >> n >> m;
9     vector<Edge> edges(m);
10    for (int i = 0; i < m; ++i)
11        cin >> edges[i].u >> edges[i].v >> edges[i].w;
12    // source 0 linked with each nodes
13    vector<ll> dist(n + 1, 0);
14
15
16    for (int i = 1; i < n; ++i) {
17        bool changed = false;
18        for (const auto& e : edges) {
19            if (dist[e.v] > dist[e.u] + e.w) {
20                dist[e.v] = dist[e.u] + e.w;
21                changed = true;
22            }
23        }
24        if (!changed)
25            break;
26    }
27
28    for (Edge e : edges)
29        if (dist[e.v] > dist[e.u] + e.w)
30            return void(cout << "-inf\n");
31
32    ll min_dist = 1e18;

```

```

33     for (int i = 1; i <= n; ++i)
34         min_dist = min(min_dist, dist[i]);
35
36     cout << min_dist << endl;
37 }

```

7.3 SPFA

```

1  const ll OO = 1e15;
2  const int N = 2500 + 5;
3  vector<pair<int, ll>> adj[N];
4  // st and par are optional, just for finding negative
5  cycles
6  vi st;
7  int n;
8  bool SPFA(int src, vector<ll> &d)
9  {
10     fill(d.begin(), d.end(), OO);
11     vi cnt(n + 1), par(n + 1, -1);
12     vector<bool> inQ(n + 1, false);
13     queue<int> q;
14     d[src] = 0;
15     q.push(src);
16     inQ[src] = true;
17     while (!q.empty())
18     {
19         int p = q.front();
20         q.pop();
21         inQ[p] = false;
22         for (auto e : adj[p])
23         {
24             int to = e.F;
25             ll w = e.S;
26             if (d[p] + w < d[to])
27             {
28                 d[to] = max(-OO, d[p] + w);
29                 par[to] = p;
30                 if (!inQ[to])
31                 {
32                     inQ[to] = true;
33                     if (++cnt[to] > n)
34                         st.pb(to);
35                     else
36                         q.push(to);
37                 }
38             }
39         }
40     }
41     sort(st.begin(), st.end());
42     st.erase(unique(st.begin(), st.end()), st.end());
43     for (auto &e : st)
44         for (int i = 0; i < n; i++)
45             e = par[e];
46     return st.empty();
47 }

```

7.4 MST (Kruskal)

```

1  DSU dsu(n);

```

```

2  vector<edge> edges;
3  for (int i = 0; i < m; i++)
4  {
5      int c, u, v;
6      cin >> u >> v >> c;
7      edges.push_back({c, u, v});
8  }
9  sort(all(edges));
10 int sum = 0;
11 for (auto &[c, u, v] : edges)
12 {
13     if (dsu.get(u, v))
14         continue;
15     sum += c;
16     dsu.merge(u, v);
17 }

```

7.5 Second Best MST

```

1  int main()
2  {
3      int n, m;
4      cin >> n >> m;
5      vector<edge> edges;
6      for (int i = 0; u, v, w; i < m; i++)
7          cin >> u >> v >> w, edges.emplace_back(edge(u, v, w, i));
8      dsu.init(n);
9      int mst = kruskal(edges); // also fills the graph with the mst
10     up[1][0] = 1;
11     mx[1][0] = INT_MIN;
12     dfs(1, 1);
13     int lca, val, u, v;
14     int secondMst = INT_MAX;
15     for (auto &edge : edges) // try all edges
16     {
17         if (usedInMst[edge.id])
18             continue;
19         u = edge.u, v = edge.v;
20         lca = LCA(u, v);
21         val = mst - max(getMax(edge.u, lca), getMax(edge.v, lca)) +
22             edge.w;
23         if (val > mst)
24             secondMst = min(secondMst, val);
25     }
26     cout << secondMst;
27 }

```

7.6 LCA

```

1  const int N = 1e5 + 5, LG = 23;
2  vector<int> graph[N];
3  int up[LG][N];
4  int deep[N]{};
5  void dfs(int cur, int par = -1, int depth = 0)
6  {
7      deep[cur] = depth;
8      for (auto &ch : graph[cur])
9      {
10         if (ch == par)

```

```

11     continue;
12     up[0][ch] = cur;
13     for(int lg = 1; lg < LG and ~up[lg - 1][ch]; lg++)
14         up[lg][ch] = up[lg - 1][up[lg - 1][ch]];
15     dfs(ch, cur, depth + 1);
16 }
17 }
18 int kthAnc(int u, int k)
19 {
20     for(int lg = 0; lg < LG and ~u; lg++)
21         if((k >> lg) & 1)
22             u = up[lg][u];
23     return u;
24 }
25 int getLca(int u, int v)
26 {
27     if(deep[v] > deep[u])
28         swap(v, u);
29     u = kthAnc(u, deep[u] - deep[v]);
30     if(u == v)
31         return u;
32     for(int lg = LG - 1; lg >= 0; --lg)
33         if(up[lg][u] != up[lg][v])
34             u = up[lg][u], v = up[lg][v];
35     return up[0][u];
36 }

```

7.7 LCA with Sparse

```

1  template <typename T>
2  struct SparseTable
3  {
4      vector<vector<T>> table;
5      vector<int> lgs;
6      function<T(T, T)> myFunction;
7
8      void build(T arr[], function<T(T, T)> fun, int n)
9      {
10         myFunction = fun;
11         table = vector<vector<T>>(n + 1, vector<T>(20));
12         lgs = vector<T>(n + 1);
13         for (int i = 0; i < n; i++)
14             table[i][0] = arr[i];
15         for (int i = 2; i <= n; i++)
16             lgs[i] = lgs[i / 2] + 1;
17         for (int lg = 1; lg <= lgs[n]; lg++)
18             for (int i = 0; i + (1 << lg) - 1 < n; i++)
19                 table[i][lg] = myFunction(table[i][lg - 1], table[i + (1 << (lg - 1))][lg -
20             1]);
21         T get(int l, int r)
22         {
23             return myFunction(table[l][lgs[r - l + 1]], table[r - (1 << lgs[r - l + 1]) + 1][lgs[r
24             - l + 1]]);
25         }
26     };
27     SparseTable<int> sp;
28     vector<int> graph[N];
29     int first[N];

```

```

29 int eular[2 * N];
30 int deep[N];
31 int timer = 0;
32 void eular_tour(int cur, int par = -1, int depth = 0)
33 {
34     deep[cur] = depth;
35     eular[timer] = cur;
36     first[cur] = timer++;
37     for (auto &ch : graph[cur])
38         if (ch != par)
39             eular_tour(ch, cur, depth + 1), eular[timer++] = cur;
40 }
41 #define getLca(x, y) sp.get(min(first[x], first[y]), max(first[x], first[y]))
42 #define getDist(x, y) deep[x] + deep[y] - ((deep[getLca(x, y)]) << 1)

```

7.8 Fast LCA

```

1  vector<int> g[N];
2  vector<pair<int, int>> queries[N]; // queries[u] contains {v, idx}
3  int ancestor[N], lca_answer[N];
4  bool visited[N];
5
6  void dfs(int u, int p, DSU &dsu) {
7      ancestor[u] = u;
8      visited[u] = true;
9
10     for (int v : g[u]) {
11         if (v == p) continue;
12         dfs(v, u, dsu);
13         dsu.merge(u, v);
14         ancestor[dsu.getPar(u)] = u;
15     }
16
17     for (auto [v, idx] : queries[u]) {
18         if (visited[v]) {
19             lca_answer[idx] = ancestor[dsu.getPar(v)];
20         }
21     }
22 }

```

7.9 HLD

```

1  struct HLD
2  {
3      int n, timer;
4      vector<int> flat, sz, tp, dep, par;
5      vector<vector<int>> graph;
6      FenwickTree bit;
7      HLD(vector<vector<int>> &graph) : graph(graph), n(graph.size() - 1)
8      {
9          timer = 0;
10         sz = flat = tp = dep = par = vector<int>(n + 1, 0);
11         preDfs(1);
12         dfs(1);
13         bit.resize(n + 1);
14         for (int i = 1; i <= n; i++)
15             update(i, 1);
16     }
17     void preDfs(int cur)

```

```

18 {
19     sz[cur] = 1;
20     for (auto &ch : graph[cur])
21     {
22         graph[ch].erase(find(all(graph[ch]), cur));
23         dep[ch] = dep[cur] + 1;
24         par[ch] = cur;
25         preDfs(ch);
26         sz[cur] += sz[ch];
27     }
28     auto __cmp = [&](int u, int v) { return (sz[u] < sz[v]); };
29     int heavy = max_element(all(graph[cur]), __cmp) - graph[cur].begin();
30     if (graph[cur].size())
31     {
32         swap(graph[cur][0], graph[cur][heavy]);
33     }
34 }
35 void dfs(int cur, int top = 1)
36 {
37     flat[cur] = ++timer, tp[cur] = top;
38     for (int i = 0; i < graph[cur].size(); i++)
39     {
40         auto &ch = graph[cur][i];
41         dfs(ch, !i ? top : ch);
42     }
43 }
44 void update(int cur, int val)
45 {
46     bit.update(flat[cur], val);
47 }
48 int query(int u, int v)
49 {
50     int ret = 0;
51     for (; tp[u] != tp[v]; u = par[tp[u]])
52     {
53         if (dep[tp[u]] < dep[tp[v]])
54             swap(u, v);
55         ret += bit.get(flat[tp[u]], flat[u]);
56     }
57     if (dep[u] < dep[v])
58         swap(u, v);
59     ret += bit.get(flat[v], flat[u]);
60     return ret;
61 }
62 };

```

7.10 Centroid Decomposition

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4  using ll = long long;
5  const int N = 2e5 + 3;
6  vector<int> g[N];
7  int sz[N];
8  int cnt[N];
9  bool isRem[N]{};
10 int n, k;
11 ll ans = 0;

```

```

12 void dfsSz(int u, int p)
13 {
14     sz[u] = 1;
15     for (auto &v : g[u])
16         if (v != p and !isRem[v])
17             dfsSz(v, u), sz[u] += sz[v];
18 }
19
20 int getCent(int u, int p, int cur_sz)
21 {
22     for (auto &v : g[u])
23         if (v != p and !isRem[v])
24             if (2 * sz[v] > cur_sz)
25                 return getCent(v, u, cur_sz);
26     return u;
27 }
28
29 void dfsAdd(int u, int p, int depth)
30 {
31     cnt[depth]++;
32     for (auto &v : g[u])
33         if (v != p and !isRem[v])
34             dfsAdd(v, u, depth + 1);
35 }
36 void getAns(int u, int p, int depth)
37 {
38     if (depth <= k)
39         ans += cnt[k - depth];
40     for (auto &v : g[u])
41         if (v != p and !isRem[v])
42             getAns(v, u, depth + 1);
43 }
44 void decompose(int u)
45 {
46     dfsSz(u, -1);
47     int cent = getCent(u, -1, sz[u]);
48
49     // Logic For Answer Here
50     cnt[0] = 1;
51     for (auto &v : g[cent])
52         if (!isRem[v])
53             getAns(v, cent, 1), dfsAdd(v, cent, 1);
54
55     // clear
56     memset(cnt, 0, sz[u] * (sizeof cnt[0]));
57
58     // Solving for other subtrees
59     isRem[cent] = 1;
60     for (auto &v : g[cent])
61         if (!isRem[v])
62             decompose(v);
63 }
64 void solve()
65 {
66     cin >> n >> k;
67     int m = n - 1;
68     while (m--)
69     {
70         int u, v;

```

```

71     cin >> u >> v, u--, v--;
72     g[u].push_back(v);
73     g[v].push_back(u);
74 }
75 decompose(0);
76 cout << ans << endl;
77 }
78
79 signed main()
80 {
81     cin.tie(0);
82     ios_base::sync_with_stdio(0);
83     int t{1};
84     // cin >> t;
85     while (t--)
86         solve();
87 }

```

7.11 DSU on Tree (SACK)

```

1  int c[N];
2  int n;
3  vector<int> graph[N];
4  int heavy[N];
5  int freq[N];
6  int subtree[N];
7  int answer = 0;
8  int ans[N];
9  void update(int val, int delta)
10 {
11     answer -= (freq[val] > 0);
12     freq[val] += delta;
13     answer += (freq[val] > 0);
14 }
15 void compress()
16 {
17     map<int, int> mp;
18     for (int i = 1; i <= n; i++)
19         if (!mp.count(c[i]))
20             mp[c[i]] = mp.size() + 1;
21     for (int i = 1; i <= n; i++)
22         c[i] = mp[c[i]];
23 }
24 void dfs0(int cur, int par)
25 {
26     for (auto &ch : graph[cur])
27     {
28         if (ch == par)
29             continue;
30         dfs0(ch, cur);
31         if (subtree[heavy[cur]] < subtree[ch])
32             heavy[cur] = ch;
33         subtree[cur] += subtree[ch];
34     }
35     subtree[cur]++;
36 }
37 void calc(int cur, int par, int delta)
38 {
39     update(c[cur], delta);

```

```

40     for (auto &ch : graph[cur])
41         if (ch != par)
42             calc(ch, cur, delta);
43 }
44 void dfs(int cur, int par, int keep)
45 {
46     for (auto &ch : graph[cur])
47     {
48         if (ch == par || ch == heavy[cur])
49             continue;
50         dfs(ch, cur, 0);
51     }
52     if (heavy[cur])
53         dfs(heavy[cur], cur, 1);
54     for (auto &ch : graph[cur])
55     {
56         if (ch == par || ch == heavy[cur])
57             continue;
58         calc(ch, cur, 1);
59     }
60     update(c[cur], 1);
61     ans[cur] = answer;
62     if (!keep)
63         calc(cur, par, -1);
64 }

```

7.12 Dsu on Tree

```

1  int c[N];
2  vector<int> graph[N];
3  set<int> s[N];
4  int ans[N];
5  void dfs(int cur, int par)
6  {
7      s[cur].insert(c[cur]);
8      for (auto &ch : graph[cur])
9      {
10         if (ch == par)
11             continue;
12         dfs(ch, cur);
13         if (s[ch].size() > s[cur].size())
14             swap(s[ch], s[cur]);
15         for (auto &x : s[ch])
16             s[cur].insert(x);
17     }
18     ans[cur] = s[cur].size();
19 }

```

7.13 Knapsack on Tree

```

1  vector<int> adj[N];
2  ll dp[N][N], val[N], sub[N];
3  int n;
4  void dfs(int node, int p)
5  {
6      sub[node] = 1;
7      vector<ll> curDP(n+5, -1e18), prvDP(n+5, -1e18);
8      prvDP[0] = 0;
9      prvDP[1] = val[node];

```



```

10  for (auto &child : adj[node]) {
11      if (child == p)
12          continue;
13      dfs(child, node);
14
15      for (int sz1 = 1; sz1 <= sub[node]; sz1++)
16          for (int sz2 = 0; sz2 <= sub[child]; sz2++)
17              curDP[sz1+sz2] = max(curDP[sz1+sz2], prvDP[sz1] + dp[child][sz2]);
18      curDP[0] = 0;
19      sub[node] += sub[child];
20      prvDP = curDP;
21  }
22
23  for (int i = 0; i <= sub[node]; i++)
24      dp[node][i] = prvDP[i];
25  }

```

7.14 Block Cut Tree

```

1  const int N = 4e5 + 3, LG = 23;
2
3  // ---- Tarjan & Block Cut Tree Part ----
4  vector<int> graph[N];
5  vector<int> BCT[N];
6  int isArt[N];
7  int low[N];
8  int dfsn[N];
9  int n, timer = 0, m, q;
10 int inBCT[N], valInBCT[N];
11
12 stack<int> st;
13 vector<vector<int>> comps;
14 bool rootEdge = false;
15 void tarjan(int cur = 1, int par = -1)
16 {
17     low[cur] = dfsn[cur] = timer++;
18     st.push(cur);
19     for (auto &ch : graph[cur])
20     {
21         if (dfsn[ch] == -1)
22         {
23             tarjan(ch, cur);
24             low[cur] = min(low[cur], low[ch]);
25             if (low[ch] >= dfsn[cur])
26             {
27                 if (dfsn[cur] == 0 and !rootEdge)
28                     rootEdge = true;
29                 else
30                     isArt[cur] = true;
31                 comps.push_back({cur});
32                 while (comps.back().back() != ch)
33                     comps.back().push_back(st.top()), st.pop();
34             }
35         }
36         else if (ch != par)
37         {
38             low[cur] = min(low[cur], dfsn[ch]);
39         }
40     }

```

```

41 }
42
43 inline void addEdgeTree(int u, int v)
44 {
45     BCT[u].push_back(v), BCT[v].push_back(u);
46 }
47
48 inline void buildBCT()
49 {
50     timer = 0;
51     for (int i = 1; i <= n; i++)
52         if (isArt[i])
53             inBCT[i] = ++timer, valInBCT[inBCT[i]] = 1;
54     for (auto &comp : comps)
55     {
56         int id = ++timer;
57         for (auto &cur : comp)
58             (isArt[cur]) ? (addEdgeTree(id, inBCT[cur])) : (inBCT[cur] = id, void());
59     }
60 }
61
62 // ---- LCA Part ----
63 int up[N][LG];
64 int deep[N];
65 int deepVal[N];
66 void dfs(int cur, int par)
67 {
68     for (auto &ch : BCT[cur])
69     {
70         if (ch == par)
71             continue;
72         up[ch][0] = cur;
73         deep[ch] = deep[cur] + 1;
74         deepVal[ch] = deepVal[cur] + valInBCT[ch];
75         for (int lg = 1; lg < LG and ~up[ch][lg - 1]; lg++)
76             up[ch][lg] = up[up[ch][lg - 1]][lg - 1];
77         dfs(ch, cur);
78     }
79 }
80 inline int kthAnc(int u, int k)
81 {
82     for (int lg = LG - 1; lg >= 0 and ~u; lg--)
83         if ((k >> lg) & 1)
84             u = up[u][lg];
85     return u;
86 }
87 inline int getLca(int u, int v)
88 {
89     if (deep[v] > deep[u])
90         swap(v, u);
91     u = kthAnc(u, deep[u] - deep[v]);
92     if (u == v)
93         return u;
94     for (int lg = LG - 1; lg >= 0; lg--)
95         if (up[u][lg] != up[v][lg])
96             u = up[u][lg], v = up[v][lg];
97     return up[u][0];
98 }

```

7.15 Tarjan Articulation Points

```

1 int tarjan(vector<vector<int>> &graph)
2 {
3     int n = graph.size() - 1;
4     vector<int> dfsn(n + 1, -1);
5     vector<int> low(n + 1, -1);
6     vector<int> isArt(n + 1, 0);
7     int timer = 0;
8     function<void(int, int)> dfs = [&](int cur, int par) {
9         low[cur] = dfsn[cur] = timer++;
10        bool rootEdge = false;
11        for (auto &ch : graph[cur])
12        {
13            if (dfsn[ch] == -1)
14            {
15                dfs(ch, cur);
16                low[cur] = min(low[cur], low[ch]);
17                if (low[ch] >= dfsn[cur])
18                {
19                    if(dfsn[cur] == 0 and !rootEdge)
20                        rootEdge = true;
21                    else
22                        isArt[cur] = true;
23                }
24            }
25            else if (ch != par)
26            {
27                low[cur] = min(low[cur], dfsn[ch]);
28            }
29        }
30    };
31    dfs(1, -1);
32    vector<int> artPoints;
33    for (int i = 1; i <= n; i++)
34        if (isArt[i])
35            artPoints.push_back(i);
36    return artPoints.size();
37 }

```

7.16 Tarjan Bridge Tree

```

1 int tarjan(vector<vector<int>> &graph)
2 {
3     int n = graph.size() - 1;
4     vector<int> dfsn(n + 1, -1), low(n + 1, -1);
5     int ret = 0;
6     int timer = 0;
7     set<pii> bridges;
8     function<void(int, int)> dfs = [&](int cur, int par) {
9         dfsn[cur] = low[cur] = timer++;
10        for (auto &ch : graph[cur])
11        {
12            if (dfsn[ch] == -1)
13            {
14                dfs(ch, cur);
15                low[cur] = min(low[cur], low[ch]);
16                if (low[ch] == dfsn[ch])

```

```

17                bridges.insert({cur, ch});
18            }
19            else if (ch != par)
20                low[cur] = min(low[cur], dfsn[ch]);
21        }
22    };
23    dfs(1, -1);
24    vector<int> comp(n + 1, -1);
25    vector<vector<int>> bridgesTree(n + 1);
26    timer = 1;
27    function<void(int)> calcComp = [&](int cur) {
28        for (auto &ch : graph[cur])
29            if (comp[ch] == -1 and !bridges.count({cur, ch}) and !bridges.count({ch, cur}))
30                comp[ch] = comp[cur], calcComp(ch);
31    };
32    for (int i = 1; i <= n; i++)
33        if (comp[i] == -1)
34            comp[i] = timer++, calcComp(i);
35    for (auto &[u, v] : bridges)
36    {
37        int cmpU = comp[u];
38        int cmpV = comp[v];
39        bridgesTree[cmpU].push_back(cmpV);
40        bridgesTree[cmpV].push_back(cmpU);
41    }
42    int mx = -1, node = -1;
43    function<void(int, int, int)> dfs2 = [&](int cur, int par, int depth)
44    {
45        if (depth > mx)
46            node = cur, mx = depth;
47        for (auto &ch : bridgesTree[cur])
48            if (ch != par)
49                dfs2(ch, cur, depth + 1);
50    };
51    dfs2(1, -1, 0);
52    mx = -1;
53    dfs2(node, -1, 0);
54    return mx;
55 }

```

7.17 Tarjan SCC

```

1 int tarjan(vector<vector<int>> &graph)
2 {
3     int n = graph.size() - 1;
4     vector<int> low(n + 1, -1), inStack(n + 1, 0), dfsn(n + 1, -1), comp(n + 1, -1);
5     vector<vector<int>> comps;
6     stack<int> st;
7     int timer = 0;
8     function<void(int)> dfs = [&](int cur) {
9         low[cur] = dfsn[cur] = timer++;
10        inStack[cur] = 1;
11        st.push(cur);
12        for (auto &ch : graph[cur])
13        {
14            if (dfsn[ch] == -1)
15            {
16                dfs(ch);
17                low[cur] = min(low[cur], low[ch]);

```

```

18     }
19     else if (inStack[ch])
20     {
21         low[cur] = min(low[cur], dfsn[ch]);
22     }
23 }
24 if (low[cur] == dfsn[cur])
25 {
26     comps.emplace_back(); // Add New Comp
27     int node = -1;
28     while (node != cur)
29     {
30         node = st.top();
31         st.pop();
32         inStack[node] = 0;
33         comps.back().push_back(node);
34         comp[node] = comps.back().size() - 1;
35     }
36 }
37
38 };
39 for (int i = 1; i <= n; i++)
40     if (dfsn[i] == -1)
41         dfs(i);
42 return (comps.size() == 1);
43 }

```

7.18 Kosaraju

```

1 int kosaraju(vector<vector<int>> &graph, vector<vector<int>> &revGraph)
2 {
3     stack<int> st;
4     int n = graph.size() - 1;
5     vector<int> visited(n + 1, 0);
6     function<void(int)> dfs1 = [&](int cur)
7     {
8         if (visited[cur])
9             return;
10        visited[cur] = 1;
11        for (auto &ch : graph[cur])
12            dfs1(ch);
13        st.push(cur);
14    };
15    function<void(int)> dfs2 = [&](int cur)
16    {
17        if (!visited[cur])
18            return;
19        visited[cur] = 0;
20        for (auto &ch : revGraph[cur])
21            dfs2(ch);
22    };
23    for (int i = 1; i <= n; i++)
24        if (!visited[i])
25            dfs1(i);
26    int sccCount = 0;
27    while (st.size())
28    {
29        int cur = st.top();
30        st.pop();

```

```

31        if (visited[cur])
32        {
33            dfs2(cur);
34            sccCount++;
35        }
36    }
37    return (sccCount == 1);
38 };

```

7.19 Check for Odd Cycle

```

1 bool CheckOddCycle()
2 {
3     int vis[N + 2];
4     memset(vis, -1, sizeof vis);
5     vis[1] = 1;
6     queue<int> q;
7     q.push(1);
8     while (!q.empty())
9     {
10        int node = q.front();
11        q.pop();
12        for (auto it : v[node])
13        {
14            if (vis[it] == -1)
15            {
16                vis[it] = 1 - vis[node];
17                q.push(it);
18            }
19            else if (vis[it] == vis[node])
20                return true;
21        }
22    }
23    return false;
24 }

```

7.20 Finding Negative Cycle

```

1 struct Edge
2 {
3     int a, b, cost;
4 };
5 int n, m;
6 vector<Edge> edges;
7 const int INF = 1000000000;
8 void solve()
9 {
10    vector<int> d(n), p(n, -1);
11    int x;
12    for (int i = 0; i < n; ++i)
13    {
14        x = -1;
15        for (Edge e : edges)
16            if (d[e.a] + e.cost < d[e.b])
17            {
18                d[e.b] = d[e.a] + e.cost;
19                p[e.b] = e.a;
20                x = e.b;
21            }

```

```

22 }
23 if (x == -1)
24     cout << "No negative cycle found.";
25 else
26 {
27     for (int i = 0; i < n; ++i)
28         x = p[x];
29     vector<int> cycle;
30     for (int v = x;; v = p[v])
31     {
32         cycle.push_back(v);
33         if (v == x && cycle.size() > 1)
34             break;
35     }
36     reverse(cycle.begin(), cycle.end());
37     cout << "Negative cycle: ";
38     for (int v : cycle)
39         cout << v << ' ';
40     cout << endl;
41 }
42 }

```

7.21 Max Flow Dinic

```

1 // O(V^2 * E)
2 struct FlowEdge
3 {
4     int v, u;
5     long long cap, flow = 0;
6
7     FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
8 };
9
10 struct Dinic
11 {
12     const long long flow_inf = 1e18;
13     vector<FlowEdge> edges;
14     vector<vector<int>> adj;
15     int n, m = 0;
16     int s, t;
17     vector<int> level, ptr;
18     queue<int> q;
19
20     Dinic(int n, int s, int t) : n(n), s(s), t(t)
21     {
22         adj.resize(n);
23         level.resize(n);
24         ptr.resize(n);
25     }
26
27     void add_edge(int v, int u, long long cap = 1)
28     {
29         edges.emplace_back(v, u, cap);
30         adj[v].push_back(m++);
31         edges.emplace_back(u, v, 0);
32         adj[u].push_back(m++);
33     }
34
35     bool bfs()

```

```

36 {
37     while (!q.empty())
38     {
39         int v = q.front();
40         q.pop();
41         for (int id : adj[v])
42         {
43             if (edges[id].cap == edges[id].flow)
44                 continue;
45             if (level[edges[id].u] != -1)
46                 continue;
47             level[edges[id].u] = level[v] + 1;
48             q.push(edges[id].u);
49         }
50     }
51     return level[t] != -1;
52 }
53
54 long long dfs(int v, long long pushed)
55 {
56     if (pushed == 0)
57         return 0;
58     if (v == t)
59         return pushed;
60     for (int &cid = ptr[v]; cid < (int)adj[v].size(); cid++)
61     {
62         int id = adj[v][cid];
63         int u = edges[id].u;
64         if (level[v] + 1 != level[u])
65             continue;
66         long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
67         if (tr == 0)
68             continue;
69         edges[id].flow += tr;
70         edges[id ^ 1].flow -= tr;
71         return tr;
72     }
73     return 0;
74 }
75
76 long long flow()
77 {
78     long long f = 0;
79     while (true)
80     {
81         fill(level.begin(), level.end(), -1);
82         level[s] = 0;
83         q.push(s);
84         if (!bfs())
85             break;
86         fill(ptr.begin(), ptr.end(), 0);
87         while (long long pushed = dfs(s, flow_inf))
88         {
89             f += pushed;
90         }
91     }
92     return f;
93 }
94

```

```

95 bool dfs_path(int v, vector<int> &cur, vector<bool> &vis)
96 {
97     cur.push_back(v);
98     vis[v] = true;
99
100     if (v == t)
101         return true;
102
103     for (int id : adj[v])
104     {
105         if ((id & 1) || edges[id].flow <= 0)
106             continue;
107
108         int u = edges[id].u;
109
110         if (!vis[u] && dfs_path(u, cur, vis))
111         {
112             edges[id].flow--;
113             return true;
114         }
115     }
116
117     cur.pop_back();
118     return false;
119 }
120
121 vector<vector<int>> paths()
122 {
123     vector<vector<int>> ret;
124     while (true)
125     {
126         vector<int> cur;
127         vector<bool> vis(n, false);
128         if (!dfs_path(s, cur, vis))
129             break;
130     }
131     ret.push_back(cur);
132 }
133 return ret;
134 }
135 }
136 };

```

7.22 MCMF (Dijkstra)

```

1 // O(F * E log V)
2 struct MCMF
3 {
4     struct Edge
5     {
6         int to;
7         int cap, flow;
8         int cost;
9         int rev;
10    };
11
12    int n;
13    vector<vector<Edge>> adj;
14    vector<int> dist;

```

```

15    vector<int> parent_edge, parent_node;
16    vector<int> pot; // Potentials array for Dijkstra
17
18    const int INF = numeric_limits<int>::max() / 2;
19
20    MCMF(int n) : n(n), adj(n), pot(n, 0)
21    {
22    }
23
24    void add_edge(int from, int to, int cap, int cost)
25    {
26        adj[from].push_back({to, cap, 0, cost, (int)adj[to].size()});
27        // Reverse edge capacity is 0, cost is negated
28        adj[to].push_back({from, 0, 0, -cost, (int)adj[from].size() - 1});
29    }
30
31    // Returns a pair of {max_flow, min_cost}
32    pair<int, int> get_mcmf(int s, int t)
33    {
34        int max_flow = 0;
35        int min_cost = 0;
36        pot.assign(n, 0); // Initialize potentials to 0 (assumes initial costs >= 0)
37
38        while (true)
39        {
40            dist.assign(n, INF);
41            parent_node.assign(n, -1);
42            parent_edge.assign(n, -1);
43
44            // Priority queue for Dijkstra: {distance, node}
45            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq
46            ;
47
48            dist[s] = 0;
49            pq.push({0, s});
50
51            while (!pq.empty())
52            {
53                auto [d, u] = pq.top();
54                pq.pop();
55
56                // Skip outdated distance pairs
57                if (d > dist[u])
58                    continue;
59
60                for (int i = 0; i < adj[u].size(); i++)
61                {
62                    Edge &e = adj[u][i];
63                    if (e.cap - e.flow > 0)
64                    {
65                        // Reduced cost calculation: cost(u,v) + pot[u] - pot[v]
66                        int reduced_cost = e.cost + pot[u] - pot[e.to];
67
68                        if (dist[e.to] > dist[u] + reduced_cost)
69                        {
70                            dist[e.to] = dist[u] + reduced_cost;
71                            parent_node[e.to] = u;
72                            parent_edge[e.to] = i;
73                            pq.push({dist[e.to], e.to});

```

```

73         }
74     }
75 }
76
77 // If we can't reach the sink, max flow is achieved
78 if (dist[t] == INF)
79     break;
80
81 // Update potentials for the next iteration
82 for (int i = 0; i < n; i++)
83 {
84     if (dist[i] != INF)
85     {
86         pot[i] += dist[i];
87     }
88 }
89
90 // Find the bottleneck capacity along the shortest path
91 int push = INF;
92 int curr = t;
93 while (curr != s)
94 {
95     int p = parent_node[curr];
96     int idx = parent_edge[curr];
97     push = min(push, adj[p][idx].cap - adj[p][idx].flow);
98     curr = p;
99 }
100
101 // Push flow and accumulate cost
102 max_flow += push;
103 // pot[t] represents the true original cost of the shortest path from s to t
104 min_cost += push * pot[t];
105
106 // Update residual graph
107 curr = t;
108 while (curr != s)
109 {
110     int p = parent_node[curr];
111     int idx = parent_edge[curr];
112     int rev_idx = adj[p][idx].rev;
113     adj[p][idx].flow += push;
114     adj[curr][rev_idx].flow -= push;
115     curr = p;
116 }
117 }
118
119 return {max_flow, min_cost};
120 }
121
122 // DFS for flow decomposition (path reconstruction)
123 int dfs_path(int v, int t, int pushed, vector<int> &path, vector<int> &edge_ptr)
124 {
125     if (v == t)
126         return pushed;
127
128     for (int &cid = edge_ptr[v]; cid < adj[v].size(); ++cid)
129     {
130         auto &edge = adj[v][cid];

```

```

132
133 // Only traverse forward edges that have positive flow
134 if (edge.flow > 0)
135 {
136     path.push_back(edge.to);
137     int tr_pushed = dfs_path(edge.to, t, min(pushed, edge.flow), path, edge_ptr);
138
139     if (tr_pushed > 0)
140     {
141         edge.flow -= tr_pushed; // Consume the flow to prevent reusing it
142         return tr_pushed;
143     }
144     path.pop_back(); // Backtrack
145 }
146 }
147 return 0;
148 }
149
150 // Reconstructs all paths taking flow from s to t
151 // Returns a vector of pairs: {bottleneck_flow, path_of_vertices}
152 // NOTE: Calling this function consumes the flow values in the graph.
153 vector<pair<int, vector<int>>> get_paths(int s, int t)
154 {
155     vector<pair<int, vector<int>>> paths;
156     vector<int> edge_ptr(n, 0); // Optimization to skip depleted edges
157
158     while (true)
159     {
160         vector<int> path;
161         path.push_back(s);
162         int pushed = dfs_path(s, t, INF, path, edge_ptr);
163
164         if (pushed == 0)
165             break;
166         paths.push_back({pushed, path});
167     }
168     return paths;
169 }
170 };

```

7.23 MCMF (SPFA)

```

1 struct Edge
2 {
3     int to;
4     int cost;
5     int cap, flow, backEdge;
6 };
7
8 struct MCMF
9 {
10     const int inf = 1000000010;
11     int n;
12     int s, t;
13     vector<vector<Edge>> g;
14     vector<int> state, pr, e;
15     vector<int> d;
16     deque<int> q;
17

```

```

18 MCMF(int n, int s, int t) : n(n), s(s), t(t)
19 {
20     g.resize(n);
21     state.resize(n);
22     pr.resize(n);
23     e.resize(n);
24     d.resize(n);
25 }
26
27 void addEdge(int u, int v, int cap, int cost)
28 {
29     Edge e1 = {v, cost, cap, 0, (int)g[v].size()};
30     Edge e2 = {u, -cost, 0, 0, (int)g[u].size()};
31     g[u].push_back(e1);
32     g[v].push_back(e2);
33 }
34
35 void bfs()
36 {
37     for (int i = 0; i < n; i++)
38         state[i] = 2, d[i] = inf, pr[i] = -1;
39     state[s] = 1;
40     q.clear();
41     q.push_back(s);
42     d[s] = 0;
43     while (!q.empty())
44     {
45         int v = q.front();
46         q.pop_front();
47         state[v] = 0;
48         for (int i = 0; i < (int)g[v].size(); i++)
49         {
50             Edge c = g[v][i];
51             if (c.flow >= c.cap || (d[c.to] <= d[v] + c.cost))
52                 continue;
53             int to = c.to;
54             d[to] = d[v] + c.cost;
55             pr[to] = v;
56             e[to] = i;
57             if (state[to] == 1)
58                 continue;
59             if (!state[to] || (!q.empty() && d[q.front()] > d[to]))
60                 q.push_front(to);
61             else
62                 q.push_back(to);
63             state[to] = 1;
64         }
65     }
66 }
67
68 pair<int, int> minCostMaxFlow()
69 {
70     int flow = 0;
71     int cost = 0;
72     while (true)
73     {
74         bfs();
75         if (d[t] == inf)
76             break;

```

```

77         int it = t, addflow = inf;
78         while (it != s)
79         {
80             addflow = min(addflow,
81                             g[pr[it]][e[it]].cap - g[pr[it]][e[it]].flow);
82             it = pr[it];
83         }
84
85         it = t;
86         while (it != s)
87         {
88             g[pr[it]][e[it]].flow += addflow;
89             g[it][g[pr[it]][e[it]].backEdge].flow -= addflow;
90             cost += g[pr[it]][e[it]].cost * addflow;
91             it = pr[it];
92         }
93         flow += addflow;
94     }
95     return {cost, flow};
96 }
97 };

```

7.24 Bipartite Matching

```

1 struct BiMatching {
2     vector<int> colAssign;
3     vector<int> vis;
4     vector<vector<int>> adjMat;
5     int n, m;
6
7     BiMatching(int n, int m) : n(n), m(m) {
8         adjMat = vector<vector<int>>(n, vector<int>(m));
9         colAssign = vector<int>(m, -1);
10    }
11
12    bool canMatch(int i) {
13        for (int j = 0; j < m; j++) {
14            if (adjMat[i][j] && !vis[j]) {
15                vis[j] = 1;
16                if (colAssign[j] == -1 || canMatch(colAssign[j])) {
17                    colAssign[j] = i;
18                    return true;
19                }
20            }
21        }
22        return false;
23    }
24
25    void addEdge(int u, int v) {
26        adjMat[u][v] = 1;
27    }
28
29    int maxMatching() {
30        int maxFlow = 0;
31        for (int i = 0; i < n; i++) {
32            vis = vector<int>(m, 0);
33            if (canMatch(i))
34                maxFlow++;
35        }

```

```

36     return maxFlow;
37 }
38 };

```

7.25 Hopcroft-Karp

```

1 struct HK
2 {
3     vector<vector<int>> adjList;
4     vector<int> BtoA;
5     HK(int n, int m)
6     {
7         adjList = vector<vector<int>>(n);
8         BtoA = vector<int>(m, -1);
9     }
10    bool dfs(int a, int L, vector<int> &A, vector<int> &B)
11    {
12        if (A[a] != L)
13            return 0;
14        A[a] = -1;
15        for (auto b : adjList[a])
16        {
17            if (B[b] == L + 1)
18            {
19                B[b] = 0;
20                if (BtoA[b] == -1 or dfs(BtoA[b], L + 1, A, B))
21                {
22                    BtoA[b] = a;
23                    return 1;
24                }
25            }
26        }
27        return 0;
28    }
29    inline void addEdge(int u, int v)
30    {
31        adjList[u].push_back(v);
32    }
33    inline int maxMatching()
34    {
35        int res = 0;
36        vector<int> A(adjList.size()), B(BtoA.size()), cur, next;
37        while (true)
38        {
39            fill(all(A), 0);
40            fill(all(B), 0);
41            cur.clear();
42            for (auto a : BtoA)
43                if (~a)
44                    A[a] = -1;
45            for (int a = 0; a < adjList.size(); a++)
46                if (!A[a])
47                    cur.push_back(a);
48            for (int lay = 1; lay++)
49            {
50                bool islast = 0;
51                next.clear();
52                for (auto a : cur)
53                    for (auto b : adjList[a])

```

```

54                {
55                    if (BtoA[b] == -1)
56                        B[b] = lay, islast = 1;
57                    else if (BtoA[b] != a and !B[b])
58                        B[b] = lay, next.push_back(BtoA[b]);
59                }
60                if (islast)
61                    break;
62                if (next.empty())
63                    return res;
64                for (auto a : next)
65                    A[a] = lay;
66                cur.swap(next);
67            }
68            for (int a = 0; a < adjList.size(); a++)
69                res += dfs(a, 0, A, B);
70        }
71    }
72 };

```

7.26 Hungarian Algorithm

```

1 int a[N][N];
2 //O(n^3)
3
4 //Hungarian Algorithm for min cost barpartide matching, works also with negative costs
5 //All the nodes on the left should have an edge with all the nodes on the right (complete
6 //graph)
7 //a[l][l] represents the cost of the edge between node l in the left and node l in the right
8 //One based and every row should be matched with exactly one column
9
10 int hung() {
11     vector<int> u(n + 1), v(m + 1), p(m + 1), way(m + 1);
12     for (int i = 1; i <= n; ++i) {
13         p[0] = i;
14         int j0 = 0;
15         vector<int> minv(m + 1, INF);
16         vector<char> used(m + 1, false);
17         do {
18             used[j0] = true;
19             int i0 = p[j0], delta = INF, j1;
20             for (int j = 1; j <= m; ++j)
21                 if (!used[j]) {
22                     int cur = a[i0][j] - u[i0] - v[j];
23                     if (cur < minv[j])
24                         minv[j] = cur, way[j] = j0;
25                     if (minv[j] < delta)
26                         delta = minv[j], j1 = j;
27                 }
28             for (int j = 0; j <= m; ++j)
29                 if (used[j])
30                     u[p[j]] += delta, v[j] -= delta;
31             else
32                 minv[j] -= delta;
33             j0 = j1;
34         } while (p[j0] != 0);
35         do {
36             int j1 = way[j0];

```



```

37         p[j0] = p[j1];
38         j0 = j1;
39     } while (j0);
40 }
41 int cost = -v[0];
42 return cost;
43 }

```

8 Strings

8.1 KMP Prefix Function

```

1 vector<int> prefix_function(string s)
2 {
3     int n = (int)s.length();
4     vector<int> pi(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = pi[i - 1];
8         while (j > 0 && s[i] != s[j])
9             j = pi[j - 1];
10        if (s[i] == s[j])
11            j++;
12        pi[i] = j;
13    }
14    return pi;
15 }
16
17 bool match(string s , string t) {
18     string cur = t + '#' + s;
19
20     vector<int> pi = prefix_function(cur);
21
22     for (int &i : pi)
23         if ( i == t.size())
24             return true;
25     return false;
26 }

```

8.2 KMP Automaton

```

1
2 const int N = 1e5 + 3;
3 int nxt[N][26];
4 void build_automaton(string& v) {
5
6     int m = v.size();
7     vector<int> pi(m);
8
9     for (int i = 1; i < m; i++) {
10         int j = pi[i-1];
11         while (j > 0 && v[i] != v[j]) j = pi[j-1];
12         if (v[i] == v[j]) j++;
13         pi[i] = j;
14     }
15
16     for (int i = 0; i <= m; i++) {

```

```

17         for (int c = 0; c < 26; c++) {
18             if (v[i] == (char)('a' + c))
19                 nxt[i][c] = i + 1;
20             else if (i > 0)
21                 nxt[i][c] = nxt[pi[i-1]][c];
22             else
23                 nxt[i][c] = 0;
24         }
25     }
26 }
27 vector<pair<int,string>>adj[N];
28 ll ans = 0;
29 int az = 0;
30 void dfs(int node, int stat){
31     for(auto &i:adj[node]){
32         int nextState = stat;
33         for(char &c:i.second)
34         {
35             nextState = nxt[nextState][c-'a'];
36             ans += (nextState == az);
37         }
38         dfs(i.first,nextState);
39     }
40 }
41
42 void solve()
43 {
44     int n;cin>>n;
45     for(int i=2,j;i<=n;i++){
46         string s;
47         cin>>j>>s;
48         adj[j].push_back({i,s});
49     }
50     string a;
51     cin>>a;
52     build_automaton(a);
53     az = a.size();
54     dfs(1,0);
55     cout<<ans<<'\n';
56 }

```

8.3 KMP Automaton 2

```

1
2 const int N = 55;
3 vector<vector<int>> build_automaton(string &v)
4 {
5     int m = v.size();
6     vector<int> pi(m + 1);
7     vector<vector<int>> nxt(26, vector<int>(N, 0));
8     for (int i = 1; i < m; i++)
9     {
10         int j = pi[i - 1];
11         while (j > 0 && v[i] != v[j])
12             j = pi[j - 1];
13         if (v[i] == v[j])
14             j++;
15         pi[i] = j;
16     }

```

```

17   for (int i = 0; i <= m; i++)
18   {
19       for (int c = 0; c < 26; c++)
20       {
21           if (v[i] == (char)('a' + c))
22               nxt[c][i] = i + 1;
23           else if (i > 0)
24               nxt[c][i] = nxt[c][pi[i - 1]];
25           else
26               nxt[c][i] = 0;
27       }
28   }
29   return nxt;
30 }
31
32 void solve()
33 {
34     string c, s, t;
35     cin >> c >> s >> t;
36     int n = c.size();
37     // mx s mn t
38     vector<vector<int>> nxtS = build_automaton(s);
39     vector<vector<int>> nxtT = build_automaton(t);
40     int ssz = s.size();
41     int tsz = t.size();
42     vector<vector<vector<int>>> dp(1004, vector<vector<int>>(53, vector<int>(53, -1e9)));
43     dp[0][0][0] = 0;
44     for (int i = 0; i < n; i++)
45     {
46         for (int j = 0; j <= ssz; j++)
47         {
48             for (int k = 0; k <= tsz; k++)
49             {
50                 if (dp[i][j][k] != -1e9)
51                 {
52                     for (int ch = 0; ch < 26; ch++)
53                     {
54                         if (c[i] == '*' || (char)('a' + ch) == c[i])
55                         {
56                             int nxt1 = nxtS[ch][j];
57                             int nxt2 = nxtT[ch][k];
58                             dp[i + 1][nxt1][nxt2] = max(dp[i + 1][nxt1][nxt2], dp[i][j][k] + (
59                                 nxt1 == ssz) - (nxt2 == tsz));
60                         }
61                     }
62                 }
63             }
64         }
65         int mx = -1e9;
66         for (int j = 0; j < 51; j++)
67         {
68             for (int k = 0; k < 51; k++)
69             {
70                 mx = max(mx, dp[n][j][k]);
71             }
72         }
73     }
74     cout << mx << '\n';

```

```

75 }

```

8.4 Z-Function

```

1 vector<int> z_function(string s)
2 {
3     int n = s.size();
4     vector<int> z(n);
5     int l = 0, r = 1;
6     for (int i = 1; i < n; i++)
7     {
8         if (i < r)
9             z[i] = min(r - i, z[i - l]);
10        while (i + z[i] < n and s[z[i]] == s[i + z[i]])
11            z[i]++;
12        if (i + z[i] > r)
13            l = i, r = i + z[i];
14    }
15    return z;
16 }

```

8.5 Manacher

```

1 vector<int> manacher(string &original)
2 {
3     int n = size(original);
4     string s = string(2 * n + 1, '#');
5     for (int i = 0; i < n; i++)
6         s[2 * i + 1] = original[i];
7     n = 2 * n + 1;
8     s.insert(s.begin(), '$');
9     s.push_back('^');
10    vector<int> p(n + 2);
11    int l = 0, r = 1;
12    for (int i = 1; i <= n; i++)
13    {
14        p[i] = min(r - i, p[l + r - i]);
15        while (s[i - p[i]] == s[i + p[i]])
16            p[i]++;
17        if (i + p[i] > r)
18            r = i + p[i], l = i - p[i];
19    }
20    return vector<int>(begin(p) + 1, end(p) - 1);
21 }

```

8.6 Hashing

```

1
2 const int mod = 1e9 + 7;
3 const int N = 1e6;
4 pair<int, int> pw[N], inv[N];
5
6 int add(int a, int b)
7 {
8     return (1ll * a + b + mod) % mod;
9 }
10 int mul(int a, int b)
11 {

```

```

12     return (1ll * a * b) % mod;
13 }
14 int fastpow(int a, int b)
15 {
16     if (!b)
17         return 1;
18     int hp = fastpow(a, b >> 1);
19     hp = mul(hp, hp);
20     return (b & 1 ? mul(hp, a) : hp);
21 }
22 int modInverse(int a)
23 {
24     return fastpow(a, mod - 2);
25 }
26 void preHash()
27 {
28     vector<int> primes;
29     for (int i = 1; i <= 293; i++)
30     {
31         bool p = true;
32         for (int j = 2; j * j <= i; j++)
33         {
34             if (i % j == 0)
35                 p = false;
36         }
37         if (p)
38             primes.push_back(i);
39     }
40     srand(time(0));
41     int b1 = primes[rand() % 62], b2 = primes[rand() % 62];
42     pw[0] = inv[0] = {1, 1};
43     int invB1 = modInverse(b1), invB2 = modInverse(b2);
44     for (int i = 1; i < N; i++)
45     {
46         pw[i] = {mul(b1, pw[i - 1].first), mul(b2, pw[i - 1].second)};
47         inv[i] = {mul(invB1, inv[i - 1].first), mul(invB2, inv[i - 1].second)};
48     }
49 }
50 struct Hash
51 {
52     vector<pair<int, int>> pre;
53     pair<int, int> h(int num, int p)
54     {
55         return {mul(num, pw[p].first), mul(num, pw[p].second)};
56     }
57     int fa7l(char a)
58     {
59         return (a - 'a' + 1);
60     }
61     Hash(string &s)
62     {
63         pre.resize(s.size());
64         for (int i = 0; i < s.size(); i++)
65         {
66             pre[i] = h(fa7l(s[i]), i);
67             if (i)
68                 pre[i] = {add(pre[i].first, pre[i - 1].first), add(pre[i].second, pre[i - 1].second)};
69         }

```

```

70     }
71     pair<int, int> getPrefix(int l, int r)
72     {
73         if (!l)
74             return pre[r];
75         return {mul(add(pre[r].first, -pre[l - 1].first), inv[l].first), mul(add(pre[r].second, -pre[l - 1].second), inv[l].second)};
76     }
77 };

```

8.7 Shorter Hash

```

1 void preHash()
2 {
3     auto now = chrono::high_resolution_clock::now();
4     auto duration = chrono::duration_cast<chrono::microseconds>(now.time_since_epoch());
5     srand(duration.count());
6     int b[] = {31, 37};
7     int pw[2][N];
8     int inv[2][N];
9     pw[0][0] = pw[1][0] = inv[0][0] = inv[1][0] = 1;
10    int invB[] = {modInverse(b[0]), modInverse(b[1])};
11    for (int j = 0; j < 2; j++)
12        for (int i = 1; i < N; i++)
13            pw[j][i] = mul(pw[j][i - 1], b[j]), inv[j][i] = mul(inv[j][i - 1], invB[j]);
14 }
15
16 void rabin()
17 {
18     for (int j = 0; j < 2; j++)
19         for (int i = 0; i < n; i++)
20             pref[i + 1][j] = addm(pref[i][j], mul(doit(s[i]), pw[j][i]));
21     function<Hash(int, int)> getHash = [&](int l, int r) -> Hash {
22         Hash h = pref[r];
23         h[0] = mul(addm(h[0], -pref[l - 1][0]), inv[0][l - 1]);
24         h[1] = mul(addm(h[1], -pref[l - 1][1]), inv[1][l - 1]);
25         return h;
26     };
27 }

```

8.8 Multiset Hash

```

1 int b[2];
2 int pw[2][N];
3 int inv[2][N];
4 void precalc()
5 {
6     b[0] = rand() + 2;
7     b[1] = rand() + 2;
8     pw[0][0] = pw[1][0] = inv[0][0] = inv[1][0] = 1;
9     int invB[] = {modInverse(b[0]), modInverse(b[1])};
10    for (int j = 0; j < 2; j++)
11        for (int i = 1; i < N; i++)
12            pw[j][i] = mul(pw[j][i - 1], b[j]), inv[j][i] = mul(inv[j][i - 1], invB[j]);
13 }
14 int doit(char c)
15 {
16     return (c - 'a' + 1);
17 }

```

```

18 typedef array<int, 2> Hash;
19 void solve()
20 {
21     int n;
22     vector<Hash> prefa(n + 1), prefb(n + 1);
23     // A Hashing
24     for (int i = 0; i < n; i++)
25         for (int j = 0; j < 2; j++)
26             prefa[i + 1][j] = addm(prefa[i][j], pw[j][a[i]]);
27     // B Hashing
28     for (int i = 0; i < n; i++)
29         for (int j = 0; j < 2; j++)
30             prefb[i + 1][j] = addm(prefb[i][j], pw[j][b[i]]);
31 }

```

8.9 Trie

```

1 struct Trie {
2     struct Node {
3         int children[26]{};
4         int f = 0;
5     };
6     vector<Node> trie;
7     Trie() { trie.emplace_back(); }
8     void insert(string &s) {
9         int node = 0;
10        for (auto &i : s) {
11            int ch = (i - 'a');
12            if (!trie[trie[node].children[ch]].f) {
13                trie[node].children[ch] = trie.size();
14                trie.emplace_back();
15            }
16            node = trie[node].children[ch];
17            trie[node].f++;
18        }
19    }
20    void erase(string &s) {
21        int node = 0;
22        for (auto &i : s) {
23            int ch = (i - 'a');
24            node = trie[node].children[ch];
25            trie[node].f--;
26        }
27    }
28    int query(string &s) {
29        int node = 0;
30        for (auto &i : s) {
31            int ch = (i - 'a');
32            if (!trie[trie[node].children[ch]].f) return 0;
33            node = trie[node].children[ch];
34        }
35        return trie[node].f;
36    }
37 };

```

8.10 Binary Trie

```

1 struct BinaryTrie
2 {

```

```

3     static constexpr int BITS = 30;
4     struct Node
5     {
6         int go[2]{};
7         int f[2]{};
8     };
9     vector<Node> trie;
10    BinaryTrie()
11    {
12        trie.emplace_back();
13    }
14    void insert(int x)
15    {
16        int cur = 0;
17        for (int i = BITS - 1; i >= 0; i--)
18        {
19            bool ch = x >> i & 1;
20            if (!trie[cur].go[ch])
21                trie[cur].go[ch] = trie.size(), trie.emplace_back();
22            trie[cur].f[ch]++;
23            cur = trie[cur].go[ch];
24        }
25    }
26    void erase(int x)
27    {
28        int cur = 0;
29        for (int i = BITS - 1; i >= 0; i--)
30        {
31            bool ch = x >> i & 1;
32            trie[cur].f[ch]--;
33            cur = trie[cur].go[ch];
34        }
35    }
36
37    int get(int x)
38    {
39        // TODO : Logic Here
40        int cur = 0;
41        int ret = 0;
42        for (int i = BITS - 1; i >= 0; i--)
43        {
44            bool ch = x >> i & 1;
45            (trie[cur].f[ch ^ 1]) ? ret |= (1 << i) : ret;
46            (trie[cur].f[ch ^ 1]) ? cur = trie[cur].go[ch ^ 1] : cur = trie[cur].go[ch];
47        }
48        return ret;
49    }
50 };

```

8.11 Aho-Corasick

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct AhoCorasick
5 {
6     static const int K = 26;
7
8     struct Node

```

```

9  {
10     int nxt[K];
11     int fail;
12
13     // Pattern IDs ending at this node
14     vector<int> ids;
15
16     // Number of times this state is visited
17     long long cnt = 0;
18
19     // Used in forbidden-pattern problems
20     bool bad = false;
21
22     Node()
23     {
24         memset(nxt, -1, sizeof(nxt));
25         fail = 0;
26     }
27 };
28
29 vector<Node> trie;
30
31 AhoCorasick()
32 {
33     trie.emplace_back(); // root
34 }
35
36 // Add pattern with ID
37 void addString(const string &s, int id)
38 {
39     int v = 0;
40
41     for (char ch : s)
42     {
43         int c = ch - 'a';
44
45         if (trie[v].nxt[c] == -1)
46         {
47             trie[v].nxt[c] = trie.size();
48             trie.emplace_back();
49         }
50
51         v = trie[v].nxt[c];
52     }
53
54     trie[v].ids.push_back(id);
55     trie[v].bad = true; // if this is a forbidden pattern
56 }
57
58 void build()
59 {
60     queue<int> q;
61
62     // Root
63     for (int c = 0; c < K; c++)
64     {
65         int u = trie[0].nxt[c];
66
67         if (u == -1)

```

```

68     {
69         trie[0].nxt[c] = 0;
70     }
71     else
72     {
73         trie[u].fail = 0;
74         q.push(u);
75     }
76 }
77
78 while (!q.empty())
79 {
80     int v = q.front();
81     q.pop();
82
83     // If failure state is bad,
84     // current state is also bad.
85     if (trie[trie[v].fail].bad)
86     {
87         trie[v].bad = true;
88     }
89
90     for (int c = 0; c < K; c++)
91     {
92
93         int u = trie[v].nxt[c];
94
95         if (u == -1)
96         {
97             // Follow failure transition
98             trie[v].nxt[c] =
99                 trie[trie[v].fail].nxt[c];
100         }
101         else
102         {
103             // Calculate failure link
104             trie[u].fail =
105                 trie[trie[v].fail].nxt[c];
106
107             q.push(u);
108         }
109     }
110 }
111 }
112
113 vector<long long> countOccurrences(const string &text, int n)
114 {
115
116     // Visit states
117     int v = 0;
118
119     for (char ch : text)
120     {
121         int c = ch - 'a';
122
123         v = trie[v].nxt[c];
124
125         trie[v].cnt++;
126     }

```

```

127
128 /*
129     Propagate counts through failure links.
130
131     If:
132         fail[v] = u
133
134     then every occurrence represented by v
135     is also an occurrence represented by u.
136 */
137
138 vector<int> order;
139 order.reserve(trie.size());
140
141 queue<int> q;
142 q.push(0);
143
144 while (!q.empty())
145 {
146     int v = q.front();
147     q.pop();
148
149     order.push_back(v);
150
151     for (int c = 0; c < K; c++)
152     {
153         int u = trie[v].nxt[c];
154
155         if (trie[u].fail == v && u != 0)
156         {
157             q.push(u);
158         }
159     }
160 }
161
162 // Reverse BFS order = deeper nodes first
163 reverse(order.begin(), order.end());
164
165 for (int v : order)
166 {
167     if (v == 0)
168         continue;
169
170     trie[trie[v].fail].cnt += trie[v].cnt;
171 }
172
173 // Answer for each pattern
174 vector<long long> ans(n);
175
176 for (int v = 0; v < (int)trie.size(); v++)
177 {
178     for (int id : trie[v].ids)
179     {
180         ans[id] = trie[v].cnt;
181     }
182 }
183
184 return ans;
185 }

```

```

186
187 ll solve(int L)
188 {
189
190     int states = trie.size();
191
192     /*
193         dp[state]
194
195         Number of valid strings of current
196         length ending in 'state'.
197     */
198
199     vector<ll> dp(states);
200     vector<ll> ndp(states);
201
202     dp[0] = 1;
203
204     for (int len = 0; len < L; len++)
205     {
206
207         fill(ndp.begin(), ndp.end(), 0);
208
209         for (int v = 0; v < states; v++)
210         {
211
212             if (dp[v] == 0)
213                 continue;
214
215             for (int c = 0; c < K; c++)
216             {
217
218                 int u = trie[v].nxt[c];
219
220                 // This creates a forbidden pattern
221                 if (trie[u].bad)
222                     continue;
223
224                 ndp[u] += dp[v];
225
226                 if (ndp[u] >= MOD)
227                     ndp[u] -= MOD;
228             }
229         }
230
231         dp.swap(ndp);
232     }
233
234     ll ans = 0;
235
236     for (int v = 0; v < states; v++)
237     {
238         ans += dp[v];
239
240         if (ans >= MOD)
241             ans -= MOD;
242     }
243
244     return ans;

```

```
245     }
246 };
```

8.12 Aho-Corasick Start/End

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4  using ll = long long;
5
6  #define int ll
7
8  struct AhoCorasick {
9      static const int K = 26;
10
11     struct Node {
12         int nxt[K];
13         int fail;
14
15         // Pattern IDs ending at this node
16         vector<int> ids;
17
18         // Number of times this state is visited
19         ll cnt = 0;
20
21         ll match_cnt = 0;
22
23         // Used in forbidden-pattern problems
24         bool bad = false;
25
26         Node() {
27             memset(nxt, -1, sizeof(nxt));
28             fail = 0;
29         }
30     };
31
32     vector<Node> trie;
33
34     AhoCorasick() {
35         trie.emplace_back(); // root
36     }
37
38     // Add pattern with ID
39     void addString(const string& s, int id) {
40         int v = 0;
41
42         for (char ch : s) {
43             int c = ch - 'a';
44
45             if (trie[v].nxt[c] == -1) {
46                 trie[v].nxt[c] = trie.size();
47                 trie.emplace_back();
48             }
49
50             v = trie[v].nxt[c];
51
52         }
53
54         trie[v].ids.push_back(id);
```

```
55         trie[v].match_cnt++;
56         trie[v].bad = true; // if this is a forbidden pattern
57     }
58
59     void build() {
60         queue<int> q;
61
62         // Root
63         for (int c = 0; c < K; c++) {
64             int u = trie[0].nxt[c];
65
66             if (u == -1) {
67                 trie[0].nxt[c] = 0;
68             } else {
69                 trie[u].fail = 0;
70                 q.push(u);
71             }
72         }
73
74         while (!q.empty()) {
75             int v = q.front();
76             q.pop();
77
78             // If failure state is bad,
79             // current state is also bad.
80             trie[v].match_cnt += trie[trie[v].fail].match_cnt;
81
82             if (trie[trie[v].fail].bad) {
83                 trie[v].bad = true;
84             }
85
86             for (int c = 0; c < K; c++) {
87
88                 int u = trie[v].nxt[c];
89
90                 if (u == -1) {
91                     // Follow failure transition
92                     trie[v].nxt[c] =
93                         trie[trie[v].fail].nxt[c];
94                 }
95                 else {
96                     // Calculate failure link
97                     trie[u].fail =
98                         trie[trie[v].fail].nxt[c];
99
100                     q.push(u);
101                 }
102             }
103         }
104     };
105
106 void solve()
107 {
108     string t;
109     cin >> t;
110
111     int n;
```

```

114     cin>>n;
115
116     AhoCorasick ac_forward;
117     AhoCorasick ac_backward;
118
119     for (int i = 0; i < n; i++) {
120         string s;
121         cin >> s;
122
123         ac_forward.addString(s,i);
124
125         reverse(s.begin(), s.end());
126         ac_backward.addString(s,i);
127     }
128
129     ac_forward.build();
130     ac_backward.build();
131
132     int len = t.length();
133     vector<int> ends_at(len, 0);
134     vector<int> starts_at(len, 0);
135
136     int v = 0;
137     for (int i = 0; i < len; i++) {
138         v = ac_forward.trie[v].nxt[t[i] - 'a'];
139         ends_at[i] = ac_forward.trie[v].match_cnt;
140     }
141
142     v = 0;
143     for (int i = len - 1; i >= 0; i--) {
144         v = ac_backward.trie[v].nxt[t[i] - 'a'];
145         starts_at[i] = ac_backward.trie[v].match_cnt;
146     }
147
148     ll tot = 0;
149     for (int i = 0; i < len - 1; i++) {
150         tot += 1LL * ends_at[i] * starts_at[i + 1];
151     }
152
153     cout << tot << "\n";
154 }
155
156 signed main()
157 {
158     ios_base::sync_with_stdio(0);
159     cin.tie(0);
160     cout.tie(0);
161
162     int t = 1;
163     // cin >> t;
164     while (t--)
165     {
166         solve();
167     }
168 }
169 }

```

8.13 Aho Matrix Trick

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll MOD = 1e9 + 7;
7
8  struct Matrix {
9
10     int n;
11
12     vector<vector<ll>> a;
13
14     Matrix(int n, bool identity = false)
15         : n(n),
16           a(n, vector<ll>(n, 0)) {
17
18         if (identity) {
19             for (int i = 0; i < n; i++) {
20                 a[i][i] = 1;
21             }
22         }
23     }
24
25     Matrix operator*(const Matrix& other) const {
26
27         Matrix res(n);
28
29         for (int i = 0; i < n; i++) {
30
31             for (int k = 0; k < n; k++) {
32
33                 if (a[i][k] == 0)
34                     continue;
35
36                 for (int j = 0; j < n; j++) {
37
38                     if (other.a[k][j] == 0)
39                         continue;
40
41                     res.a[i][j] +=
42                         a[i][k] * other.a[k][j];
43
44                     res.a[i][j] %= MOD;
45                 }
46             }
47         }
48
49         return res;
50     }
51 };
52
53 // =====
54 // Matrix exponentiation
55 // =====
56
57 Matrix power(Matrix base, long long exp) {
58
59

```



```

60 Matrix res(base.n, true);
61
62 while (exp > 0) {
63     if (exp & 1) {
64         res = res * base;
65     }
66
67     base = base * base;
68
69     exp >>= 1;
70 }
71
72 return res;
73 }
74
75 // =====
76 // Aho-Corasick
77 // =====
78
79 struct AhoCorasick {
80
81     static const int K = 26;
82
83     struct Node {
84
85         int nxt[K];
86
87         int fail;
88
89         // Is this state forbidden?
90         bool bad;
91
92         Node() {
93             memset(nxt, -1, sizeof(nxt));
94
95             fail = 0;
96             bad = false;
97         }
98     };
99
100     vector<Node> trie;
101
102     AhoCorasick() {
103         trie.emplace_back();
104     }
105
106     // -----
107     // Add forbidden pattern
108     // -----
109
110     void addString(const string& s) {
111         int v = 0;
112
113         for (char ch : s) {

```

```

119         int c = ch - 'a';
120
121         if (trie[v].nxt[c] == -1) {
122             trie[v].nxt[c] =
123                 trie.size();
124             trie.emplace_back();
125         }
126
127         v = trie[v].nxt[c];
128     }
129
130     trie[v].bad = true;
131 }
132
133 // -----
134 // Build automaton
135 // -----
136
137 void build() {
138     queue<int> q;
139
140     // Root
141     for (int c = 0; c < K; c++) {
142         int u = trie[0].nxt[c];
143
144         if (u == -1) {
145             trie[0].nxt[c] = 0;
146         } else {
147             trie[u].fail = 0;
148             q.push(u);
149         }
150     }
151
152     // BFS
153     while (!q.empty()) {
154         int v = q.front();
155         q.pop();
156
157         // If failure state is bad,
158         // this state is also bad.
159         if (trie[trie[v].fail].bad) {
160             trie[v].bad = true;
161         }
162
163         for (int c = 0; c < K; c++) {
164             int u = trie[v].nxt[c];

```

```

178         if (u == -1) {
179             trie[v].nxt[c] =
180                 trie[trie[v].fail].nxt[c];
181         } else {
182             trie[u].fail =
183                 trie[trie[v].fail].nxt[c];
184             q.push(u);
185         }
186     }
187 }
188
189 // -----
190 // Build transition matrix
191 // -----
192
193 Matrix getMatrix() {
194     int states = trie.size();
195     Matrix M(states);
196     for (int v = 0; v < states; v++) {
197         // We never use bad states.
198         if (trie[v].bad)
199             continue;
200         for (int c = 0; c < K; c++) {
201             int u = trie[v].nxt[c];
202             // Don't transition into a bad state.
203             if (trie[u].bad)
204                 continue;
205             /*
206              v --character--> u
207              Therefore:
208              M[v][u]++
209              ++ is important because multiple
210              characters can lead to the same
211              pair of states.
212              */
213             M.a[v][u]++;
214             if (M.a[v][u] >= MOD)
215                 M.a[v][u] -= MOD;
216         }
217     }
218     return M;
219 }

```

```

237     }
238
239     // -----
240     // Solve
241     // -----
242
243     ll solve(long long L) {
244         int states = trie.size();
245         Matrix M = getMatrix();
246         // M^L
247         Matrix P = power(M, L);
248         /*
249          Initially:
250          dp = [1, 0, 0, ...]
251          Therefore after L characters:
252          dp[j] = P[0][j]
253          */
254         ll ans = 0;
255         for (int state = 0; state < states; state++) {
256             // Bad states are not reachable anyway,
257             // but skipping them makes the meaning clear.
258             if (trie[state].bad)
259                 continue;
260             ans += P.a[0][state];
261             if (ans >= MOD)
262                 ans -= MOD;
263         }
264         return ans;
265     }
266 };
267
268 int main() {
269     ios::sync_with_stdio(false);
270     cin.tie(nullptr);
271     int n;
272     long long L;
273     cin >> n >> L;
274     AhoCorasick ac;
275     for (int i = 0; i < n; i++) {

```

```

296
297     string s;
298
299     cin >> s;
300
301     ac.addString(s);
302 }
303
304 ac.build();
305
306 cout << ac.solve(L) << '\n';
307
308 return 0;
309 }

```

8.14 Hard Hash LCA

```

1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4 #include <vector>
5 #define Kero
6
7 ios_base::sync_with_stdio(0);
8
9 cout.tie(0);
10
11 cin.tie(0)
12 #define ll long long
13 #define ull unsigned long long
14 #define all(x) x.begin(), x.end()
15 #define allr(x) x.rbegin(), x.rend()
16 #define int ll
17 #define ld long double
18 #define pii pair<int, int>
19 #define endl '\n'
20 #define see(x) " [" << #x << " = " << (x) << "]"
21 using namespace std;
22 using namespace __gnu_pbds;
23 template <typename T>
24 using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
25
26 template <typename T>
27 using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag, tree_order_statistics_node_update>;
28
29 // order_of_key(k) : Number of items strictly smaller than k
30 // find_by_order(k) : K-th element in a set (counting from zero)
31 const int INF = 1e18, N = 2e5 + 5, mod = 1e9 + 7, LG = 20;
32 const ld pi = 2 * acos(0), eps = 1e-6;
33
34 void fileIO()
35 {
36 #ifndef ONLINE_JUDGE
37     freopen("io/input.txt", "r", stdin);
38     freopen("io/output.txt", "w", stdout);
39 #endif
40 }
41
42 // Coding is so easy if you simulate on paper first

```

```

38 map<int, int> facts;
39 int addm(int a)
40 {
41     return ((a % mod) + mod) % mod;
42 }
43 template <typename... Args>
44 int addm(int a, Args... args)
45 {
46     return (addm(a) + addm(args...)) % mod;
47 }
48 int subtm(int a, int b)
49 {
50     return ((a % mod) - (b % mod) + mod) % mod;
51 }
52 int mul(int a)
53 {
54     return ((a % mod) + mod) % mod;
55 }
56 template <typename... Args>
57 int mul(int a, Args... args)
58 {
59     return (mul(a) * mul(args...)) % mod;
60 }
61 int fastpow(int b, int p)
62 {
63     if (p == 0 || b == 1)
64         return 1;
65     int sub = fastpow(b, p / 2);
66     if (p % 2 == 0)
67         return mul(sub, sub);
68     return mul(mul(sub, sub), b);
69 }
70 inline int modInverse(int a)
71 {
72     return fastpow(a, mod - 2);
73 }
74 inline int divi(int a, int b)
75 {
76     return mul(a, modInverse(b));
77 }
78 ll fact(ll a)
79 {
80     if (a == 0ll)
81         return 1;
82     if (!facts.count(a))
83         facts[a] = mul(fact(a - 1), a);
84     return facts[a];
85 }
86 inline int nPr(int n, int r)
87 {
88     if (n < r)
89         return 0;
90     return divi(fact(n), fact(n - r));
91 }
92 inline int nCr(int n, int r)
93 {
94     if (n < r)
95         return 0;
96     return divi(nPr(n, r), fact(r));

```

```

97 }
98 inline int starsNbars(int n, int r)
99 {
100     return nCr(n + r - 1, r);
101 }
102 int b[] = {31, 37};
103 int pw[N][2];
104 int inv[N][2];
105 inline void precalc()
106 {
107     pw[0][0] = pw[0][1] = inv[0][0] = inv[0][1] = 1;
108     int invB[] = {modInverse(b[0]), modInverse(b[1])};
109     for (int i = 1; i < N; i++)
110         for (int j = 0; j < 2; j++)
111             pw[i][j] = mul(pw[i - 1][j], b[j]), inv[i][j] = mul(inv[i - 1][j], invB[j]);
112 }
113 inline int doit(char c)
114 {
115     return (c - 'a' + 1);
116 }
117 struct Hash
118 {
119     int sum[2];
120     int len;
121     Hash()
122     {
123         sum[0] = sum[1] = 0;
124         len = 0;
125     }
126     Hash(char c)
127     {
128         sum[0] = doit(c) * b[0];
129         sum[1] = doit(c) * b[1];
130         len = 1;
131     }
132     Hash operator+(const Hash &other) const
133     {
134         Hash ret = *this;
135         ret.len = len + other.len;
136         for (int i = 0; i < 2; i++)
137         {
138             ret.sum[i] = mul(ret.sum[i], pw[other.len][i]);
139             ret.sum[i] = addm(ret.sum[i], other.sum[i]);
140         }
141         return ret;
142     }
143     // subtract The Suffix
144     Hash operator-(const Hash &other) const
145     {
146         Hash ret = *this;
147         ret.len = len - other.len;
148         for (int i = 0; i < 2; i++)
149         {
150             ret.sum[i] = addm(ret.sum[i], -other.sum[i]);
151             ret.sum[i] = mul(ret.sum[i], inv[other.len][i]);
152         }
153         return ret;
154     }
155     bool operator==(const Hash &other) const

```

```

156 {
157     return len == other.len and sum[0] == other.sum[0] and sum[1] == other.sum[1];
158 }
159 Hash sub_pref(const Hash &other) const
160 {
161     Hash ret = *this;
162     ret.len = len - other.len;
163     for (int i = 0; i < 2; i++)
164     {
165         ret.sum[i] = addm(ret.sum[i], -mul(other.sum[i], pw[ret.len][i]));
166     }
167     return ret;
168 }
169 static vector<Hash> getHash(string &str)
170 {
171     vector<Hash> ret;
172     ret[0] = Hash(str[0]);
173     for (int i = 1; i < str.size(); i++)
174         ret[i] = ret[i - 1] + Hash(str[i]);
175     return ret;
176 }
177 static Hash getRange(vector<Hash> &hsh, int l, int r)
178 {
179     l--;
180     Hash ret = hsh[r];
181     if (l)
182         ret = ret - hsh[l];
183     return ret;
184 }
185 };
186
187 string str;
188 int deep[N];
189 Hash deepHash[N];
190 Hash deepHash2[N];
191 vector<int> g[N];
192 int up[N][LG];
193 void init(int n)
194 {
195     for (int i = 0; i <= n; i++)
196     {
197         memset(up[i], 0, sizeof up[i]);
198         deepHash[i] = Hash();
199         deepHash2[i] = Hash();
200         g[i].clear();
201     }
202 }
203 void dfs(int cur)
204 {
205     for (auto &ch : g[cur])
206     {
207         if (ch == up[cur][0])
208             continue;
209         deep[ch] = deep[cur] + 1;
210         up[ch][0] = cur;
211         deepHash[ch] = deepHash[cur] + Hash(str[ch]);
212         deepHash2[ch] = Hash(str[ch]) + deepHash2[cur];
213         for (int lg = 1; lg < LG and up[ch][lg - 1]; lg++)

```

```

215         up[ch][lg] = up[up[ch][lg - 1]][lg - 1];
216     dfs(ch);
217 }
218 }
219 inline int kthAnc(int u, int k)
220 {
221     for (int lg = LG - 1; lg >= 0; lg--)
222         if ((k >> lg) & 1)
223             u = up[u][lg];
224     return u;
225 }
226 inline int getLca(int u, int v)
227 {
228     if (deep[v] > deep[u])
229         swap(v, u);
230     u = kthAnc(u, deep[u] - deep[v]);
231     if (u == v)
232         return u;
233     for (int lg = LG - 1; lg >= 0; lg--)
234         if (up[u][lg] != up[v][lg])
235             u = up[u][lg], v = up[v][lg];
236     return up[u][0];
237 }
238 inline int getDist(int u, int v)
239 {
240     return deep[u] + deep[v] - 2 * deep[getLca(u, v)];
241 }
242 inline Hash getPath(int u, int v)
243 {
244     int lca = getLca(u, v);
245     Hash hsh_v = deepHash[v].sub_pref(deepHash[up[lca][0]]);
246     Hash hsh_u = deepHash2[u] - deepHash2[lca];
247     // cout << see(lca) << endl;
248     // cout << hsh_u.sum[0] << endl;
249     return hsh_u + hsh_v;
250 }
251 inline int kthInPath(int u, int v, int k)
252 {
253     int lca = getLca(u, v);
254     int dist = deep[u] + deep[v] - 2 * deep[lca];
255     if (deep[u] - deep[lca] < k)
256         return kthAnc(v, dist - k);
257     else
258         return kthAnc(u, k);
259 }
260 inline int query(int a, int b, int c, int d)
261 {
262     int d1 = getDist(a, b);
263     int d2 = getDist(c, d);
264     int dist = min(d1, d2);
265     // cout << getPath(a, b).sum[0] << endl;
266     // cout << getPath(c, d).sum[0] << endl;
267     int l = 0, r = dist, mid, ans = -1;
268     while (l <= r)
269     {
270         mid = l + (r - l) / 2;
271         Hash hsh_a = getPath(a, kthInPath(a, b, mid));
272         Hash hsh_c = getPath(c, kthInPath(c, d, mid));
273         if (hsh_a == hsh_c)

```

```

274     {
275         l = mid + 1;
276     }
277     else
278     {
279         ans = mid;
280         r = mid - 1;
281     }
282 }
283 if (ans == -1)
284 {
285     if (d1 > d2)
286         return 1;
287     else if (d1 < d2)
288         return 2;
289     return 0;
290 }
291 int kth_a = kthInPath(a, b, ans);
292 int kth_c = kthInPath(c, d, ans);
293 if (str[kth_a] > str[kth_c])
294     return 1;
295     return 2;
296 }
297 inline void solve()
298 {
299     int n;
300     cin >> n;
301     init(n);
302     // Hash hsh;
303     // str = "bab";
304     // for (int i = 0; i < 3; i++)
305     //     hsh = hsh + Hash(str[i]);
306     // cout << hsh.sum[0] << endl;
307     // hsh = hsh - Hash();
308     // cout << hsh.sum[0] << endl;
309     // cout << endl;
310     cin >> str;
311     str = " " + str;
312     for (int i = 2, u, v; i <= n; i++)
313         cin >> u >> v, g[u].push_back(v), g[v].push_back(u);
314     deep[1] = 0;
315     deepHash[1] = Hash(str[1]);
316     deepHash2[1] = Hash(str[1]);
317     dfs(1);
318     // cout << getPath(1, 3).sum[0] << endl;
319     int q;
320     cin >> q;
321     while (q--) {
322         int a, b, c, d;
323         cin >> a >> b >> c >> d;
324         cout << query(a, b, c, d) << endl;
325     }
326 }
327
328 signed main()
329 {
330     Kero;
331     fileIO();
332     precalc();

```

```

333     int t = 1;
334     cin >> t;
335     while (t-->
336         solve();
337     return 0;
338 }

```

9 Math & Geometry

9.1 Max Manhattan Distance

- For a set of points: $\max(|\max(u) - \min(u)|, |\max(v) - \min(v)|)$
- Where $u = x + y$ and $v = x - y$

9.2 Linear Sieve

```

1  const int N = 10000000;
2  vector<int> lp(N+1);
3  vector<int> pr;
4
5  for (int i=2; i <= N; ++i) {
6      if (lp[i] == 0) {
7          lp[i] = i; pr.push_back(i);
8      }
9      for (int j = 0; i * pr[j] <= N; ++j)
10         {
11             lp[i * pr[j]] = pr[j];
12             if (pr[j] == lp[i])
13                 break; }
14     }
15 }

```

9.3 Miller-Rabin Primality

```

1  using u64 = uint64_t;
2  using u128 = __uint128_t;
3  u64 binpower(u64 base, u64 e, u64 mod)
4  {
5      u64 result = 1;
6      base %= mod;
7      while (e)
8      {
9          if (e & 1)
10             result = (u128)result * base % mod;
11             base = (u128)base * base % mod;
12             e >>= 1;
13     }
14     return result;
15 }
16 bool check_composite(u64 n, u64 a, u64 d, int s)
17 {
18     u64 x = binpower(a, d, n);
19     if (x == 1 || x == n - 1)
20         return false;
21     for (int r = 1; r < s; r++)
22     {
23         x = (u128)x * x % n;

```

```

24         if (x == n - 1)
25             return false;
26     }
27     return true;
28 }
29 bool MillerRabin(u64 n)
30 {
31     // returns true if n is prime, else returns false.
32     if (n < 2)
33         return false;
34     int r = 0;
35     u64 d = n - 1;
36     while ((d & 1) == 0)
37     {
38         d >>= 1;
39         r++;
40     }
41     for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
42     {
43         if (n == a)
44             return true;
45         if (check_composite(n, a, d, r))
46             return false;
47     }
48     return true;
49 }

```

9.4 Phi & Mobius Sieve

```

1  int phi[N + 1];
2  int mu[N + 1];
3  bool vis[N + 1];
4  vector<int> divs[N + 1];
5
6  void pre()
7  {
8      for (int i = 1; i <= N; i++)
9          phi[i] = i, mu[i] = 1;
10     for (int i = 2; i <= N; i++)
11         if (phi[i] == i)
12             for (int j = i; j <= N; j += i)
13                 phi[j] -= phi[j] / i;
14     for (int i = 2; i <= N; i++)
15         if (!vis[i])
16             for (int j = i; j <= N; j += i)
17                 vis[j] = 1, mu[j] = (j % (i * i) == 0 ? 0 : (mu[j] * -1));
18 }

```

9.5 Phi / Sqrt

```

1  int phi(int x)
2  {
3      int ret = x;
4      for (int i = 2; i * i <= x; i++)
5      {
6          if (x % i == 0)
7          {
8              while (x % i == 0)
9                  x /= i;

```

```

10     ret -= ret / i;
11 }
12 }
13 if (x > 1)
14     ret -= ret / x;
15 return ret;
16 }

```

9.6 Extended Euclidean

```

1 int euclid(int a, int b, int &x, int &y)
2 {
3     if (a < 0 || b < 0)
4     {
5         int g = euclid(abs(a), abs(b), x, y);
6         if (a < 0)
7             x *= -1;
8         if (b < 0)
9             y *= -1;
10        return g;
11    }
12    if (b == 0)
13    {
14        x = 1, y = 0;
15        return a;
16    }
17    int x1, y1;
18    int g = euclid(b, a % b, x1, y1);
19    x = y1;
20    y = x1 - y1 * (a / b);
21    return g;
22 }
23 int find_sol(int a, int b, int c, int &x0, int &y0, bool &found)
24 {
25     int g = euclid(abs(a), abs(b), x0, y0);
26     found = 1;
27     if (c % g)
28     {
29         found = 0;
30         return g;
31     }
32     x0 *= c / g;
33     y0 *= c / g;
34     return g;
35 }

```

9.7 Linear Diophantine

```

1 void shift_solution(int &x, int &y, int a, int b, int cnt)
2 {
3     x += cnt * b;
4     y -= cnt * a;
5 }
6 int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy)
7 {
8     int x, y, g;
9     if (!find_any_solution(a, b, c, x, y, g))
10        return 0;
11    a /= g;

```

```

12    b /= g;
13    int sign_a = a > 0 ? +1 : -1;
14    int sign_b = b > 0 ? +1 : -1;
15    shift_solution(x, y, a, b, (minx - x) / b);
16    if (x < minx)
17        shift_solution(x, y, a, b, sign_b);
18    if (x > maxx)
19        return 0;
20    int lx1 = x;
21    shift_solution(x, y, a, b, (maxx - x) / b);
22    if (x > maxx)
23        shift_solution(x, y, a, b, -sign_b);
24    int rx1 = x;
25    shift_solution(x, y, a, b, -(miny - y) / a);
26    if (y < miny)
27        shift_solution(x, y, a, b, -sign_a);
28    if (y > maxy)
29        return 0;
30    int lx2 = x;
31    shift_solution(x, y, a, b, -(maxy - y) / a);
32    if (y > maxy)
33        shift_solution(x, y, a, b, sign_a);
34    int rx2 = x;
35    if (lx2 > rx2)
36        swap(lx2, rx2);
37    int lx = max(lx1, lx2);
38    int rx = min(rx1, rx2);
39    if (lx > rx)
40        return 0;
41    return (rx - lx) / abs(b) + 1;
42 }

```

9.8 Find All Diophantine Solutions

```

1 int gcd(int a, int b, int &x, int &y)
2 {
3     if (b == 0)
4     {
5         x = 1;
6         y = 0;
7         return a;
8     }
9     int x1, y1;
10    int d = gcd(b, a % b, x1, y1);
11    x = y1;
12    y = x1 - y1 * (a / b);
13    return d;
14 }
15 bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g)
16 {
17     g = gcd(abs(a), abs(b), x0, y0);
18     if (c % g)
19     {
20         return false;
21     }
22     x0 *= c / g;
23     y0 *= c / g;
24     if (a < 0)
25         x0 = -x0;

```

```

26     if (b < 0)
27         y0 = -y0;
28     return true;
29 }
30 void shift_solution(int &x, int &y, int a, int b, int cnt)
31 {
32     x += cnt * b;
33     y -= cnt * a;
34 }
35 int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy)
36 {
37     int x, y, g;
38     if (!find_any_solution(a, b, c, x, y, g))
39         return 0;
40     a /= g;
41     b /= g;
42     int sign_a = a > 0 ? +1 : -1;
43     int sign_b = b > 0 ? +1 : -1;
44     shift_solution(x, y, a, b, (minx - x) / b);
45     if (x < minx)
46         shift_solution(x, y, a, b, sign_b);
47     if (x > maxx)
48         return 0;
49     int lx1 = x;
50     shift_solution(x, y, a, b, (maxx - x) / b);
51     if (x > maxx)
52         shift_solution(x, y, a, b, -sign_b);
53     int rx1 = x;
54     shift_solution(x, y, a, b, -(miny - y) / a);
55     if (y < miny)
56         shift_solution(x, y, a, b, -sign_a);
57     if (y > maxy)
58         return 0;
59     int lx2 = x;
60     shift_solution(x, y, a, b, -(maxy - y) / a);
61     if (y > maxy)
62         shift_solution(x, y, a, b, sign_a);
63     int rx2 = x;
64     if (lx2 > rx2)
65         swap(lx2, rx2);
66     int lx = max(lx1, lx2);
67     int rx = min(rx1, rx2);
68     if (lx > rx)
69         return 0;
70     return (rx - lx) / abs(b) + 1;
71 }

```

9.9 GCD Trick

```

1  int n, m, k;
2  cin >> n >> m >> k;
3
4  // f[x] = frequency of value x in the array
5  vector<ll> f(m + 1);
6
7  for (int i = 0; i < n; i++)
8  {
9      int x;
10     cin >> x;

```

```

11     f[x]++;
12 }
13
14 /*
15     div[i] = number of elements divisible by i.
16
17     We iterate over multiples of i:
18
19         i, 2i, 3i, ...
20
21     and add their frequencies.
22 */
23 vector<ll> div(m + 1);
24
25 for (int i = 1; i <= m; i++)
26 {
27     for (int j = i; j <= m; j += i)
28     {
29         div[i] += f[j];
30     }
31 }
32
33 /*
34     g[i] = number of NON-EMPTY subsequences
35           whose gcd is exactly i.
36
37     First:
38
39         2^div[i] - 1
40
41     counts all non-empty subsequences where every
42     selected element is divisible by i.
43
44     Their gcd can be:
45
46         i, 2i, 3i, ...
47
48     Therefore we subtract the answers for all
49     proper multiples of i.
50 */
51 vector<ll> g(m + 1);
52
53 for (int i = m; i >= 1; i--)
54 {
55
56     // All non-empty subsequences consisting of
57     // elements divisible by i.
58     g[i] = (modPow(2, div[i]) - 1 + MOD) % MOD;
59
60     /*
61         Remove subsequences whose gcd is a proper
62         multiple of i.
63     */
64     for (int j = 2 * i; j <= m; j += i)
65     {
66         g[i] -= g[j];
67
68         if (g[i] < 0)
69

```



```

70         g[i] += MOD;
71     }
72 }

```

9.10 Count Number of Divisors

```

1 bool is_prime_square(int n)
2 {
3     int num = sqrtl(n);
4     return num * num == n && MillerRabin(num);
5 }
6 int count_divisors(int n)
7 {
8     // can also be used to get:
9     // sum and product of the divisors
10    int ans = 1;
11    for (auto &p : primes)
12    {
13        if (p * p * p > n)
14            break;
15        if (n % p != 0)
16            continue;
17        int cnt = 0;
18        while (n % p == 0)
19        {
20            cnt++;
21            n /= p;
22        }
23        ans *= (cnt + 1);
24    }
25    if (MillerRabin(n))
26        ans *= 2;
27    else if (is_prime_square(n))
28        ans *= 3;
29    else if (n != 1)
30        ans *= 4;
31    return ans;
32 }

```

9.11 Count Subarrays with LCM = q

```

1 //count all arrays with lcm(a[i], a[i+1]) == k
2
3 inline vector<int> getPrimeFactors(int x)
4 {
5     map<int, int> freq;
6     vector<int> ret;
7     for (int i = 2; i * i <= x; i++)
8         while (x % i == 0)
9             freq[i]++, x /= i;
10    if (x > 1)
11        freq[x]++;
12    for (auto &[p, frq] : freq)
13        ret.push_back(frq);
14    return ret;
15 }
16 void solve()
17 {
18     int n, k;

```

```

19     cin >> n >> k;
20     vector<int> factors = getPrimeFactors(k);
21     ll ans = 1;
22     Matrix S, T;
23     for (auto &frq : factors)
24     {
25         S = {{0, 1}};
26         T = {{0, 1},
27              {frq, 1}};
28         T = power(T, n);
29         S = mul(S, T);
30         S[0][1] = (1LL * S[0][0] + S[0][1]) % mod;
31         ans *= S[0][1];
32         ans %= mod;
33     }
34     cout << ans << endl;
35 }

```

9.12 Counting (Combinations)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5 const ll MOD = 1e9 + 7;
6 const int MAXN = 1e6 + 5; // adjust as needed
7
8 ll fact[MAXN], invfact[MAXN];
9
10 // Fast Power (a^b % MOD)
11 ll fastPow(ll a, ll b) {
12     ll res = 1;
13     a %= MOD;
14     while (b) {
15         if (b & 1) res = res * a % MOD;
16         a = a * a % MOD;
17         b >>= 1;
18     }
19     return res;
20 }
21
22 // Modular Inverse (MOD must be prime)
23 ll modinv(ll a) {
24     return fastPow(a, MOD - 2);
25 }
26
27 // Precompute factorials and inverse factorials
28 void init_factorials() {
29     fact[0] = 1;
30     for (int i = 1; i < MAXN; i++)
31         fact[i] = fact[i - 1] * i % MOD;
32
33     invfact[MAXN - 1] = modinv(fact[MAXN - 1]);
34     for (int i = MAXN - 2; i >= 0; i--)
35         invfact[i] = invfact[i + 1] * (i + 1) % MOD;
36 }
37
38 // nCr (Combination)

```

```

40 ll nCr(ll n, ll r) {
41     if (r < 0 || r > n) return 0;
42     return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
43 }
44 bool nCr(int n, int r)
45 {
46     return ((n & r) == r);
47 }
48
49 // nPr (Permutation)
50 ll nPr(ll n, ll r) {
51     if (r < 0 || r > n) return 0;
52     return fact[n] * invfact[n - r] % MOD;
53 }
54
55 // Combination with Repetition
56 // (n multichoose r) = (n+r-1)Cr
57 ll nCr_rep(ll n, ll r) {
58     return nCr(n + r - 1, r);
59 }
60
61 // Stars and Bars
62 // Distribute k identical items into n distinct boxes
63 ll stars_and_bars(ll n, ll k) {
64     // (k + n - 1)C(n - 1)
65     return nCr(k + n - 1, n - 1);
66 }
67
68 // Stars and Bars (at least 1 in each box)
69 ll stars_and_bars_at_least_one(ll n, ll k) {
70     // Give 1 to each first
71     if (k < n) return 0;
72     return nCr(k - 1, n - 1);
73 }

```

9.13 Matrix Power

```

1  const int mod = 1e9 + 7;
2  using Row = vector<long long>;
3  using Matrix = vector<Row>;
4
5  Matrix mul(Matrix &a, Matrix &b) {
6      int n = a.size(), m = a[0].size(), k = b[0].size();
7      Matrix res(n, Row(k));
8      for(int i = 0; i < n; ++i)
9          for(int j = 0; j < k; ++j)
10             for(int o = 0; o < m; ++o) {
11                 res[i][j] += 1ll * a[i][o] * b[o][j] % mod;
12                 if(res[i][j] >= mod) res[i][j] -= mod;
13                 if(res[i][j] < 0) res[i][j] += mod;
14             }
15     return res;
16 }
17
18 Matrix power(Matrix a, long long b) {
19     int n = a.size();
20     Matrix res(n, Row(n));
21     for(int i = 0; i < n; ++i) res[i][i] = 1;
22

```

```

23     while(b) {
24         if(b&1) res = mul(res, a);
25         a = mul(a, a), b >>= 1;
26     }
27
28     return res;
29 }

```

9.14 Big Integer

```

1  __int128 read()
2  {
3      __int128 x = 0, f = 1;
4      char ch = getchar();
5      while (ch < '0' || ch > '9')
6      {
7          if (ch == '-')
8              f = -1;
9          ch = getchar();
10     }
11     while (ch >= '0' && ch <= '9')
12     {
13         x = x * 10 + ch - '0';
14         ch = getchar();
15     }
16     return x * f;
17 }
18
19 void print(__int128 x)
20 {
21     if (x < 0)
22     {
23         putchar('-');
24         x = -x;
25     }
26     if (x > 9)
27         print(x / 10);
28     putchar(x % 10 + '0');
29 }
30
31 bool cmp(__int128 x, __int128 y) {
32     return x > y;
33 }

```

9.15 Solve Quadratics

```

1  int quadRoots(double a, double b, double c, pair<double, double> &out)
2  {
3      assert(a != 0);
4      double disc = b * b - 4 * a * c;
5      if (disc < 0)
6          return 0;
7      double sum = (b >= 0) ? -b - sqrt(disc) : -b + sqrt(disc);
8      out = {sum / (2 * a), sum == 0 ? 0 : (2 * c) / sum};
9      return 1 + (disc > 0);
10 }

```

9.16 Convex Hull

```

1 struct point
2 {
3     ll x, y;
4     point(ll x=0, ll y=0) : x(x), y(y)
5     {
6     }
7     point operator-(point other)
8     {
9         return point(x - other.x, y - other.y);
10    }
11    bool operator<(const point &other) const
12    {
13        return x != other.x ? x < other.x : y < other.y;
14    }
15 };
16 ll cross(point a, point b)
17 {
18     return a.x * b.y - a.y * b.x;
19 }
20 ll dot(point a, point b)
21 {
22     return a.x * b.x + a.y * b.y;
23 }
24 struct sortCCW
25 {
26     point center;
27
28     sortCCW(point center) : center(center)
29     {
30     }
31     bool operator()(point a, point b)
32     {
33         ll res = cross(a - center, b - center);
34         if (res)
35             return res > 0;
36         return dot(a - center, a - center) < dot(b - center, b - center);
37     }
38 };
39 vector<point> hull(vector<point> v)
40 {
41     sort(v.begin(), v.end());
42     sort(v.begin() + 1, v.end(), sortCCW(v[0]));
43     v.push_back(v[0]);
44     vector<point> ans;
45     for (auto i : v)
46     {
47         int sz = ans.size();
48         while (sz > 1 && cross(i - ans[sz - 1], ans[sz - 2] - ans[sz - 1]) <= 0)
49             ans.pop_back(), sz--;
50         ans.push_back(i);
51     }
52     ans.pop_back();
53     return ans;
54 }

```

9.17 Geometry

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #define T long double
5 #define point complex<T>
6 #define Y imag()
7 #define X real()
8 /*
9 abs(): Returns the magnitude (absolute value) of the complex number.
10 arg(): Returns the argument (polar angle) of the complex number.
11 conj(): Returns the complex conjugate of the complex number
12 hypot(c,d): Takes two lengths and returns the hypotenuse
13 polar(c,theta): Return a complex with these polar cords
14 */
15 const double EPS = 1e-9;
16 const double PI = 3.14159265358979323846;
17 point mkvec(point a, point b){
18     return b - a;
19 }
20
21 T cross(point a, point b) {
22     return a.X*b.Y - a.Y*b.X;
23 }
24
25 T clamp(T val){
26     return min((T)1.0 , max( (T)-1.0 , val));
27 }
28
29 T dot(point a, point b) {
30     return (conj(a) * b).real();
31 }
32 // dotProduct = |A| * |B| * cos(Theta)
33 T calcAngle(point v1, point v2){
34     return acos(clamp(dot(v1,v2)/abs(v1)/abs(v2)));
35 }
36 // Convert degrees to radians
37 T toRad(T deg) {
38     return deg * PI / 180.0;
39 }
40
41 // Rotation CCW (use negative angle for CW)
42 point rotate(point p, point pivot, T angle) {
43     return (p - pivot) * polar((T)1.0, angle) + pivot;
44 }
45 /*
46 // Rotation CCW (use negative angle for CW)
47 point rotate(point p, point pivot, int ang) {
48     p = (p - pivot);
49     if(ang == 90)
50         p = point(-p.Y, p.X);
51     else if(ang == -90) {
52         p = point(p.Y, -p.X);
53     }
54     return p + pivot;
55 }
56 */
57
58 point readPoint(){
59     int x,y;

```

```

60     cin>>x>>y;
61     return point(x,y);
62 }
63
64 T angleCCW(point v1, point v2) {
65     T a1 = atan2(v1.Y, v1.X);
66     T a2 = atan2(v2.Y, v2.X);
67     T angle = a2 - a1;
68
69     if (angle < -EPS) angle += 2.0 * PI;
70     return angle;
71 }
72
73 // 1 - left - ccw
74 int orientation (point a,point b,point c){
75     point ab = mkvec(a,b);
76     point ac = mkvec(a,c);
77     T cr = cross(ab,ac);
78     if(cr > 0) return 1;
79     if(cr < 0) return -1;
80     return 0;
81 }
82
83
84 ////////////////Lines/////////////////
85
86 // normal equation => Ax + By + C = 0
87 // slop-intercept form => y = Mx + B
88 // Essential Core Functions
89 //double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
90 //double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
91 //double magnitude(Point a) { return hypot(a.x, a.y); }
92 //double dist(Point a, Point b) { return hypot(a.x - b.x, a.y - b.y); }
93 // summation of angles for regular n-Gon
94 // In degrees: (n - 2) x 180°
95
96 // =====
97 // 2. LINE STRUCT (Ax + By + C = 0)
98 // =====
99 struct Line {
100     double a, b, c;
101     double slope,yinter;
102     Line(double a = 0, double b = 0, double c = 0) : a(a), b(b), c(c) {}
103
104     // Build line from two points
105     void fromTwoPoints(point p1, point p2) {
106         a = p1.Y - p2.Y;
107         b = p2.X - p1.X;
108         c = -a * p1.X - b * p1.Y;
109     }
110
111     // Build line from a point and a normal vector
112     void fromPointAndNormal(point p, point normal) {
113         a = normal.X;
114         b = normal.Y;
115         c = -a * p.X - b * p.Y;
116     }
117 }
118 void slop_inter(){

```

```

119     slope = a/b;
120     yinter = c/b;
121 }
122 };
123
124 // =====
125 // 3. MULTI-FUNCTION UTILITIES
126 // =====
127
128 // Perpendicular distance from Point to Line
129 T distToLine(point p, Line l) {
130     return abs(l.a * p.X + l.b * p.Y + l.c) / hypot(l.a, l.b);
131 }
132
133 // Line-Line Intersection
134 // Returns false if lines are parallel/coincident, true if they intersect
135 bool intersect(Line l1, Line l2, point& out_intersection) {
136     T zn = l1.a * l2.b - l2.a * l1.b;
137     if (abs(zn) < EPS) return false;
138
139     out_intersection = point (- (l1.c * l2.b - l2.c * l1.b) / zn, -(l1.a * l2.c - l2.a * l1.c)
140                               / zn);
141     return true;
142 }
143
144 // Area of any simple polygon using Shoelace Formula
145 T polygonArea(const vector<point>& vertices) {
146     T area = 0;
147     int n = vertices.size();
148     for (int i = 0; i < n; i++) {
149         area += cross(vertices[i], vertices[(i + 1) % n]);
150     }
151     return abs(area) / 2.0;
152 }
153 // T t = dot(mkvec(a,p),mkvec(a,b)) / dot(mkvec(a,b),mkvec(a,b));
154 // t < 0 -> less than a
155 // t > 1 -> more than b
156 // else the image is on the line
157 bool onSegment(point a, point b, point c) {
158     return (c.X >= min(a.X, b.X) && c.X <= max(a.X, b.X) &&
159            c.Y >= min(a.Y, b.Y) && c.Y <= max(a.Y, b.Y));
160 }
161 // CIRCLES//
162 T circumcircleRadius(point p1, point p2, point p3) {
163     T a = abs(p1 - p2);
164     T b = abs(p2 - p3);
165     T c = abs(p3 - p1);
166
167     T area = polygonArea({p1, p2, p3});
168
169     // Avoid division by zero if the points are collinear (on the same line)
170     if (area < EPS) return -1.0;
171
172     return (a * b * c) / (4.0 * area);
173 }
174
175 vector<point> intersectCircles(point c1, T r1, point c2, T r2) {
176     T d = abs(c2 - c1);

```

```

177
178 // Case 1: Circles are too far apart, or one is strictly inside the other
179 if (d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS || d < EPS) {
180     return {}; // No intersection
181 }
182
183 // Case 2: They intersect or are tangent
184 // Using Law of Cosines to find the angle from c1: r2^2 = r1^2 + d^2 - 2*r1*d*cos(a)
185 T cos_a = (r1 * r1 + d * d - r2 * r2) / (2.0 * r1 * d);
186 cos_a = clamp(cos_a); // Ensure value is between [-1, 1] before acos
187 T angle = acos(cos_a);
188
189 // Create a vector from c1 pointing exactly to c2, then scale its length to r1
190 point v = (c2 - c1) / d * r1;
191
192 point p1 = c1 + v * polar((T)1.0, angle);
193 point p2 = c1 + v * polar((T)1.0, -angle);
194
195 if (abs(p1 - p2) < EPS)
196     return {p1};
197
198 return {p1, p2};
199 }
200
201 //not tested
202 // Returns the center of the circle passing through 3 non-collinear points (Circumcenter)
203 point circumcenter(point a, point b, point c) {
204     point ab = b - a; // Vector from A to B
205     point ac = c - a; // Vector from A to C
206
207     // Calculate the determinant (which is 2 * cross product)
208     T D = 2.0 * cross(ab, ac);
209
210     // If D is 0, it means the 3 points are collinear (on the same line),
211     // so a circle cannot be drawn through them.
212     if (abs(D) < EPS) {
213         return point(1e18, 1e18); // Return a dummy/infinity point as an error flag
214     }
215
216     // In <complex>, std::norm(v) returns the SQUARED magnitude (x^2 + y^2)
217     // Don't confuse it with abs() which returns the actual distance.
218     T ab_sq = norm(ab);
219     T ac_sq = norm(ac);
220
221     // Apply Cramer's rule to find the center relative to 'a'
222     T cx = (ac.Y * ab_sq - ab.Y * ac_sq) / D;
223     T cy = (ab.X * ac_sq - ac.X * ab_sq) / D;
224
225     // Add 'a' back to shift the center to its true absolute position
226     return a + point(cx, cy);
227 }
228
229 int pointInPolygon(vector<point> polygon, point p){
230     int n = polygon.size();
231
232     for(int i=0; i<n; i++){
233         if( (orientation(polygon[i], polygon[(i+1)%n], p) == 0) && onSegment(polygon[i],
234             polygon[(i+1)%n], p))

```

```

235         // "BOUNDARY\n";
236         return 1;
237     }
238 }
239
240 bool inside = false;
241 for(int i=0; i<n; i++){
242     point a = polygon[i], b = polygon[(i+1)%n];
243     if(a.Y > b.Y)
244         swap(a, b);
245     if( p.Y >= a.Y && p.Y < b.Y )
246     {
247         if(orientation(a, b, p) == 1){
248             inside = !inside;
249         }
250     }
251 }
252
253 // cout << (inside? "INSIDE\n": "OUTSIDE\n");
254 if(inside)
255     return 2;
256 return 0;
257 }
258
259 // line xy , ab
260 bool lineSegmentsIntersect(point a, point b, point x, point y){
261     long long r1 = orientation(a, b, x);
262     long long r2 = orientation(a, b, y);
263     long long r3 = orientation(x, y, a);
264     long long r4 = orientation(x, y, b);
265
266     if(r1*r2 < 0 && r3*r4 < 0)
267         return 1;
268     else if( (r1 == 0 && onSegment(a, b, x)) ||
269         (r2 == 0 && onSegment(a, b, y)) ||
270         (r3 == 0 && onSegment(x, y, a)) ||
271         (r4 == 0 && onSegment(x, y, b))
272     ){
273         return 1;
274     }
275     return 0;
276 }
277
278
279 // check if the all angles are 90 in quad
280 bool four90(vector<point> v){
281     for(int i=0; i<4; i++){
282         if(dot(v[(i+1)%4] - v[i], v[(i+2)%4] - v[(i+1)%4]) != 0)
283             return false;
284     }
285     return true;
286 }
287
288 int main()
289 {
290     ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
291
292
293

```

```

294 function<bool>(point,point,point,point)> formSquare = [](point a,point b,point c,point d)
    -> bool{
295     vector<T>dis;
296     dis.push_back(norm(a-b));
297     dis.push_back(norm(a-c));
298     dis.push_back(norm(a-d));
299     dis.push_back(norm(d-b));
300     dis.push_back(norm(d-c));
301     dis.push_back(norm(b-c));
302     sort(dis.begin(),dis.end());
303     return dis[0] == dis[1] && dis[1] == dis[2] && dis[2] == dis[3] && dis[4] == dis[5] &&
        dis[0] > EPS;
304 };
305
306 return 0;
307 }

```

10 Dynamic Programming

10.1 LIS

```

1 int n, mx = 0;
2 int arr[N] = {};
3 vector<int> dp(N, INF);
4 void solve()
5 {
6     // memset(dp, -1, sizeof dp);
7     cin >> n;
8     for (size_t i = 0; i < n; i++)
9     {
10         cin >> arr[i];
11     }
12     dp[0] = -INF;
13     for (int i = 0; i < n; i++)
14     {
15         int l = upper_bound(dp.begin(), dp.end(), arr[i]) - dp.begin();
16         if (dp[l - 1] < arr[i] && arr[i] < dp[l])
17             dp[l] = arr[i];
18     }
19     for (int l = 0; l <= n; l++)
20     {
21         if (dp[l] < INF)
22             mx = l;
23     }
24     cout << mx << endl;
25 }

```

10.2 LIS with DS

```

1 void solve()
2 {
3     int n, k;
4     cin >> n >> k;
5     vector<int> v(n);
6     int mx = 0;
7     for (auto &x : v)
8         cin >> x, mx = max(mx, x);

```

```

9     vector<BIT> dp(k + 1, BIT(mx + 1));
10
11     for (int i = 0; i < n; i++)
12     {
13         dp[1].update(v[i] + 1, 1);
14         for (int j = 1; j < k; j++)
15         {
16             int add = dp[j].pref(v[i]);
17             dp[j + 1].update(v[i] + 1, add);
18         }
19     }
20     cout << dp[k].pref(mx + 1);
21 }

```

10.3 Bitset DP

```

1 class Solution {
2 public:
3
4     int maxTotalReward(vector<int>& a) {
5         int n = a.size();
6         a.push_back(-1);
7         sort(a.begin(), a.end());
8         const int MX = 1e5;
9         bitset<MX> dp, allones;
10        dp[0] = 1;
11        int j = 0;
12        for(int i = 1; i <= n; i++)
13        {
14            // for(int j = 0; j < MX; j++)
15            // {
16            //     dp[i][j] = dp[i - 1][j];
17            //     if(2 * a[i] > j and j - a[i]>=0 )
18            //         dp[i][j] = max(dp[i][j], dp[i - 1][j - a[i]] + a[i]);
19            // }
20            while(a[i] != j)
21                allones[j++] = 1;
22            dp |= (dp & allones) << a[i];
23        }
24        for(int i = MX - 1; ~i; i--)
25            if(dp[i])
26                return i;
27        return 0;
28    }
29 };

```

10.4 Digit DP

```

1 const int mod = 998244353;
2 ll add(ll a, ll b) {
3     return (a + b + mod) % mod;
4 }
5 ll mul(ll a, ll b) {
6     return (a * b) % mod;
7 }
8 struct Node{
9     ll cnt,sum;
10 };
11 bool memo[2][2][(1<<10)][20];

```

```

12 Node dp [2][2][1<10]][20]; //ru,rd,lz,msk (taken digits) , i = n
13 string l,r;
14 int n,k;
15
16 ll pow10[20];
17 Node rec(bool ru,bool rd,int msk,int i){
18
19     if(i == n){
20         //
21         return {1,0};
22     }
23
24     Node &ret = dp[ru][rd][msk][i];
25     if(memo[ru][rd][msk][i])
26         return ret;
27
28     short up = ru ? r[i] - '0' : 9;
29     short down = rd ? l[i] - '0' : 0;
30
31     ret = { 0 , 0};
32
33     for(ll d=down ; d<=up ; d++){
34         ll nexMsk = msk;
35         if( !(msk == 0 && d == 0)) // not leading zeros
36             nexMsk = msk | (1<<d);
37
38         if( __builtin_popcount(nexMsk) <= k ){
39             Node temp = rec(ru && d==up , rd && d==down , nexMsk ,i+1);
40
41             ret.cnt = add(ret.cnt , temp.cnt);
42             ll dig_val = mul(mul(d , pow10[n-i-1]) , temp.cnt );
43             ret.sum = add ( add(ret.sum,dig_val) , temp.sum);
44         }
45     }
46     memo[ru][rd][msk][i] = true;
47     return ret;
48 }
49 void solve() {
50     cin>>l>>r >> k;
51     n = r.size();
52     while(l.size()!=n)
53         l = "0" + l;
54
55     memset(dp,-1,sizeof dp);
56
57     pow10[0] = 1;
58     for(int i=1;i<20;i++)
59         pow10[i] = mul(pow10[i-1],10);
60
61     cout<<rec(1,1,0,0).sum <<'\n';
62 }

```

10.5 Digit DP (Kero)

```

1 void solve()
2 {
3     string l, r;
4     int k;
5     cin >> l >> r >> k;

```

```

6     if(r.size() > l.size())
7         l = string(r.size() - l.size(), '0') + l;
8     k = min(k, 99ll);
9     int dp[r.size()][2][2][100][100];
10    int n = r.size();
11    memset(dp, -1, sizeof(dp));
12    function<int(int, int , int , int, int)> sol = [&](int i, int canExceed, int canDown, int
    remNum, int remSum)->int
13    {
14        if(i == n)
15            return !(remSum or remNum);
16        auto &ret = dp[i][canExceed][canDown][remNum][remSum];
17        if(~ret)
18            return ret;
19        ret = 0;
20        for(int d = 0; d < 10; d++)
21        {
22            if(!canExceed and d > (r[i] - '0'))
23                continue;
24            if(!canDown and d < (l[i] - '0'))
25                continue;
26            ret += sol(i + 1, canExceed | (d != r[i] - '0'), canDown | (d != l[i] - '0'), (
    remNum + (d * (int)powl(10, n - i - 1))) % k, (remSum + d) % k);
27        }
28        return ret;
29    };
30    cout << sol(0, 0, 0, 0, 0) << endl;
31
32 }

```

10.6 Count Distinct Subsequences

```

1 dp[0] = 1;
2 // Traverse through all lengths from 1 to n.
3 for (int i = 1; i <= n; i++)
4 {
5     dp[i] = (2 * dp[i - 1]) % md;
6     // If current character has appeared
7     // before, then remove all subsequences
8     // ending with previous occurrence.
9     if (last[str[i - 1]] != -1)
10         dp[i] = (dp[i] - dp[last[str[i - 1]]] + md) % md;
11     // Mark occurrence of current character
12     last[str[i - 1]] = (i - 1);
13 }
14 return dp[n]; // dp[n] - 1 to remove empty subsequence

```

10.7 Largest Zero Submatrix

```

1 int zero_matrix(vector<vector<int>> a)
2 {
3     int n = a.size();
4     int m = a[0].size();
5     int ans = 0;
6     vector<int> d(m, -1), d1(m), d2(m);
7     stack<int> st;
8     for (int i = 0; i < n; ++i)
9     {
10         for (int j = 0; j < m; ++j)

```

```

11 {
12     if (a[i][j] == 1)
13         d[j] = i;
14 }
15 for (int j = 0; j < m; ++j)
16 {
17     while (!st.empty() && d[st.top()] <= d[j])
18         st.pop();
19     d1[j] = st.empty() ? -1 : st.top();
20     st.push(j);
21 }
22 while (!st.empty())
23     st.pop();
24 for (int j = m - 1; j >= 0; --j)
25 {
26     while (!st.empty() && d[st.top()] <= d[j])
27         st.pop();
28     d2[j] = st.empty() ? m : st.top();
29     st.push(j);
30 }
31 while (!st.empty())
32     st.pop();
33 for (int j = 0; j < m; ++j)
34     ans = max(ans, (i - d[j]) * (d2[j] - d1[j] - 1));
35 }
36 return ans;
37 }

```

10.8 Number of Paths of Length K

```

1 #include <bits/stdc++.h>
2 #define Mazen_Alaa ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
3 typedef long long ll;
4 typedef long double ld;
5
6 using namespace std;
7
8
9 void solve() {
10     int n,m,k;cin>>n>>m>>k;
11
12     Matrix T(n+1,Row(n+1));
13     // if u is parent to v => t[u][v] = 1;
14     for(int i=0,u,v;i<m;i++){
15         cin>>u>>v;
16         T[u][v] = 1;
17     }
18     Row a(n+1);
19     for(ll &i:a)
20         i = 1;
21     Matrix v;
22     v.push_back(a);
23     T = power(T,k);
24     Matrix res = mul(v,T);
25
26     ll tot = 0;
27     for(int i=1;i<=n;i++){
28         tot += res[0][i];
29         if(tot >= mod)

```

```

30         tot -= mod;
31         if(tot < 0)
32             tot += mod;
33     }
34     cout<<tot<<'\n';
35 }

```

10.9 Sub-Rects with Unique Values

```

1 vector<vector<int>>> height(n, vector<int>(m, 1)); // num of elements top of a[i]
2 [j] that equal to it
3     vector<vector<int>>>
4         dp(n, vector<int>(m, 0)); // num of subrectangles that
5 a[i][j] is its bottom right corner
6     // setting height for each element
7     for (int i = 1; i < n; i++)
8     {
9         for (int j = 0; j < m; j++)
10         {
11             if (a[i][j] == a[i - 1][j])
12                 height[i][j] += height[i - 1][j];
13         }
14     }
15     for (int i = 0; i < n; i++)
16     {
17         vector<int> st;
18         for (int j = 0; j < m; j++)
19         {
20             int k = -1;
21             while (st.size())
22             {
23                 int col = st.back();
24                 if (a[i][col] != a[i][j] || height[i][col] < height[i][j])
25                     break;
26                 st.pop_back();
27             }
28             // if k = -1: means that all cols behind: same element and their height > height[i][
29 j]
30             // if k = val: means that all columns between [k+1:j] are valid
31             // k = val because of: a[i][k]!=a[i][j] or height[i][k] < height[i][j]
32             if (st.size())
33                 k = st.back();
34             // each col in [k+1:j] has at least height[i][j], so all of them can end
35 at a[i][j] as bot - right corner
36             dp[i][j] = height[i][j] * (j - k);
37             if (k != -1)
38             {
39                 // here: k = val because of height[i][k] < height[i][j]
40                 // so all subrectangles that ends at i,k -> can end at i,j. As
41 height[i][k] < height[i][j] if (a[i][j] == a[i][k])
42                 dp[i][j] += dp[i][k];
43             }
44             st.push_back(j);
45         }
46     }

```


11 Game Theory

11.1 Grundy (Sprague-Grundy)

```

1  const int H = 1e4 + 5;
2  vector<int> grundy(H, 0);
3  for (int i = 1; i < H; i++)
4  {
5      set<int> reachable;
6      for (auto &move : moves)
7          if (i - move >= 0)
8              reachable.insert(grundy[i - move]);
9
10     for (auto &x : reachable)
11         if (grundy[i] == x)
12             grundy[i]++;
13     else
14         break;
15 }

```

11.2 Mex Set

```

1  int sz;
2  struct Set
3  {
4      set<int> missing;
5      Set()
6      {
7          for(int i = 0; i < sz; i++)
8              {
9                  missing.insert(i);
10             }
11     }
12     int getMex()
13     {
14         return missing.empty() ? sz : *missing.begin();
15     }
16     void insert(int x)
17     {
18         if(missing.count(x))
19             missing.erase(x);
20     }
21     void clear()
22     {
23         for(int i = 0; i < sz; i++)
24             {
25                 missing.insert(i);
26             }
27     }
28 };

```

12 Miscellaneous

12.1 Baby Step Giant Step

```

1  int d_log(int a, int b, int mod) {

```

```

2      int n = sqrt(mod) + 1;
3      // store b * a ^ q for all 0 <= q < n
4      hash_map<int, int> mp;
5      for (int i = 0, cur = b; i <= n; i++, cur = 1LL * cur * a % mod)
6          mp[cur] = i;
7      int an = fp(a, n, mod);
8      // lets try all a ^ (n * i) == b * a ^ q for all q values using the hash map
9      for (int i = 1, cur = an; i <= n; i++, cur = 1LL * cur * an % mod)
10         if (mp.find(cur) != mp.end())
11             return n * i - mp[cur];
12     return -1;
13 }

```

12.2 Catalan Numbers

```

1  /*
2  * =====
3  * CATALAN NUMBERS: CLASSIC SUB-PROBLEMS
4  * The nth Catalan number (C_n) solves several classic combinatorial problems:
5  *
6  * 1. Valid Parentheses (Dyck Words):
7  *   Number of correctly matched bracket sequences consisting of 'n' pairs
8  *   of parentheses.
9  *   Example (n=3): ((())), ()(()), ()()(), (()()), (()()) -> C_3 = 5
10 *
11 * 2. Binary Search Trees (BSTs):
12 *   Number of structurally unique BSTs that can store 'n' distinct keys.
13 *
14 * 3. Polygon Triangulation:
15 *   Number of ways to divide a convex polygon with 'n+2' sides into triangles
16 *   by drawing non-intersecting diagonals.
17 *
18 * 4. Grid Paths (Dyck Paths):
19 *   Number of monotonic paths on an n x n grid from bottom-left (0,0) to
20 *   top-right (n,n) that never cross above the main diagonal.
21 *
22 * 5. Non-crossing Chords:
23 *   Number of ways to connect 2n points on a circle to form 'n' chords
24 *   such that no two chords intersect.
25 *
26 * 6. Full Binary Trees:
27 *   Number of full binary trees (where every vertex has either 0 or 2 children)
28 *   that have exactly 'n' internal nodes (and n+1 leaves).
29 * =====
30 */
31
32 long long catalanMod(int n) {
33     if (n <= 1) return 1;
34
35     long long numerator = 1;
36     long long denominator = 1;
37
38     // Calculate (2n)! / (n! * n!) % MOD
39     for (int i = 0; i < n; i++) {
40         numerator = (numerator * (2LL * n - i)) % MOD;
41         denominator = (denominator * (i + 1)) % MOD;
42     }
43
44     // Multiply numerator by the modular inverse of the denominator

```

```

45     long long combinations = (numerator * modInverse(denominator)) % MOD;
46
47     // Multiply by the modular inverse of (n + 1)
48     long long result = (combinations * modInverse(n + 1)) % MOD;
49
50     return result;
51 }
52
53 long long catalan(int n) {
54     if (n <= 1) return 1;
55
56     long long res = 1;
57
58     // Calculate nCr(2n, n) safely
59     // Note: This relies on exact divisibility at each step to avoid early truncation
60     for (int i = 0; i < n; ++i) {
61         res *= (2LL * n - i);
62         res /= (i + 1);
63     }
64
65     // Divide by (n + 1) to get the Catalan number
66     return res / (n + 1);
67 }

```

12.3 Compress

```

1 vector<int> temp = a;
2 sort(temp.begin(), temp.end());
3 temp.erase(unique(temp.begin(), temp.end()), temp.end());
4
5
6 for(int i = 0; i < n; i++){
7     a[i] = lower_bound(temp.begin(), temp.end(), a[i]) - temp.begin() + 1;
8 }

```

12.4 Math Operations on Strings

```

1 string multiply(const string &a, const string &b)
2 {
3     if (a.size() < b.size())
4         swap(a, b);
5     int n = a.size(), m = b.size();
6     vector<int> ans(n + m + 1);
7     for (int i = n - 1; ~i; i--)
8     {
9         for (int j = m - 1; ~j; j--)
10        {
11            int idx = i + j;
12            int cur = (a[i] - '0') * (b[j] - '0');
13            ans[idx] += cur;
14            if (ans[idx] >= 10)
15            {
16                if (idx)
17                {
18                    ans[idx - 1] += (ans[idx] / 10);
19                    ans[idx] %= 10;
20                }
21            }
22        }
23    }
24 }

```

```

23 }
24 string ret;
25 int q = 0;
26 if (ans[0] >= 10)
27 {
28     q = 1;
29     ret += char('0' + ans[0] / 10);
30     ans[0] %= 10;
31 }
32 int cnt = 1;
33 for (int i = 0; i < n + m - 1; i++)
34 {
35     if (q || ans[i] != 0)
36     {
37         for (int j = i; j < n + m - 1; j++)
38             ret += char('0' + ans[j]);
39         cnt = 0;
40         break;
41     }
42 }
43 if (cnt)
44     ret = "0";
45 return ret;
46 }
47
48 string stringStringAddition(string a, string b)
49 {
50     string ret, other;
51     if (a.size() > b.size())
52         ret = a, other = b;
53     else
54         ret = b, other = a;
55     reverse(ret.begin(), ret.end());
56     reverse(other.begin(), other.end());
57     int diff = ret.size() - other.size();
58     other += string(diff, '0');
59     for (int i = ret.size() - 1; i >= 0; i--)
60     {
61         int cur = (ret[i] - '0') + (other[i] - '0');
62         ret[i] = (cur % 10) + '0';
63         cur /= 10;
64         int currentIdx = i;
65         while (cur > 0)
66         {
67             currentIdx++;
68             if (currentIdx == ret.size())
69                 ret += '0';
70             cur += (ret[currentIdx] - '0');
71             ret[currentIdx] = (cur % 10) + '0';
72             cur /= 10;
73         }
74     }
75     reverse(ret.begin(), ret.end());
76     return ret;
77 }
78
79 string longDivision(string num, ll divisor)
80 {
81     string ans;

```

```

82     ll idx = 0;
83     ll temp = num[idx] - '0';
84     while (temp < divisor)
85         temp = temp * 10 + (num[++idx] - '0');
86     while (num.size() > idx)
87     {
88         ans += (temp / divisor) + '0';
89         temp = (temp % divisor) * 10 + num[++idx] -
90             '0';
91     }
92     if (ans.length() == 0)
93         return "0";
94     return ans;
95 }

```

12.5 Ternary Search

```

1  while (s <= e)
2  {
3      m1 = s + (e - s) / 3;
4      m2 = e - (e - s) / 3;
5      int ans1 = solve(m1);
6      int ans2 = solve(m2);
7      ans = min({ans, ans1, ans2});
8      if (ans1 > ans2)
9          s = m1 + 1;
10     else
11         e = m2 - 1;
12 }

```

12.6 2D Partial Sum

```

1  void update(int x1, int y1, int x2, int y2, int val)
2  {
3      p_sum[x1][y1] += val;
4      p_sum[x2 + 1][y2 + 1] += val;
5      p_sum[x1][y2 + 1] -= val;
6      p_sum[x2 + 1][y1] -= val;
7  }
8
9  for (int i = 1; i < 1004; i++)
10 {
11     for (int j = 1; j < 1004; j++)
12     {
13         pref[i][j] += pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1];
14     }
15 }
16 while (q--)
17 {
18     ll la, lb, pa, pb;
19     cin >> la >> lb >> pa >> pb;
20     cout << pref[pa - 1][pb - 1] - pref[la][pb - 1] - pref[pa - 1][lb] + pref[la][lb] << endl;
21 }

```

12.7 Stress Test

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  // Random test case generator (writes input to in.txt)
6  void generate_test_case()
7  {
8      ofstream fout("io/input.txt");
9      // TODO : Test Generation
10     fout.close();
11 }
12
13 // Read entire file into a string (for output comparison)
14 string read_file(const string &filename)
15 {
16     ifstream fin(filename);
17     stringstream buffer;
18     buffer << fin.rdbuf();
19     return buffer.str();
20 }
21
22 int main()
23 {
24     srand(time(0));
25     int test_num = 0;
26
27     for(; test_num < 1000;)
28     {
29         test_num++;
30         generate_test_case();
31
32         // Run both solutions on the same input
33         system("./build/naive");
34         system("./build/fast");
35         // Compare outputs
36         string out = read_file("io/output.txt");
37         string brute = read_file("io/output_naive.txt");
38
39         if (out != brute)
40         {
41             cout << "Test " << test_num << " FAILED!\n";
42             // cout << "Input:\n" << read_file("io/input.txt") << "\n";
43             // cout << "Fast output:\n" << out << "\n";
44             // cout << "Brute output:\n" << brute << "\n";
45             break;
46         }
47         else
48         {
49             cout << "Test " << test_num << " passed.\n";
50         }
51     }
52
53     return 0;
54 }

```